

HOCHSCHULE OSNABRÜCK
UNIVERSITY OF APPLIED SCIENCES

Fakultät
Ingenieurwissenschaften und Informatik

Hausarbeit im Fach
Objektorientierte Analyse
Und Design
Card Game Framework mit Java

Autoren:	Akim Zdorovtsov
	1108288
	Lukas Ochs
	1109497
	Stanislav Zholob
	1127974

Abgabedatum:	03.08.2026
---------------------	------------

Inhaltsverzeichnis

1.	Einleitung	1
1.1	Aufgabenstellung (Akim)	1
1.2	Aufbau der Hausarbeit (Akim)	1
2	Grundlagen	3
2.1	State Pattern (Lukas)	3
2.2	Factory Pattern (Akim)	3
2.3	Command Pattern (Akim)	4
2.4	Observer Pattern (Lukas)	4
2.5	Strategy Pattern (Akim)	4
3	Anforderungsanalyse	6
3.1	Glossar (Akim)	6
3.2	Stakeholder und Ziele (Akim)	6
3.3	Use-Case-Diagramm (Akim)	7
3.4	Use-Case-Dokumentation (Akim)	8
3.5	Rupp Schablone (Lukas)	9
3.6	Aktivitätsdiagramm (Lukas)	10
4	Modellierung	11
4.1	Grundarchitektur des Frameworks (Stanislav)	11
4.2	State Pattern (Phasen) (Lukas)	12
4.3	Command Pattern (Undo) (Akim)	13
4.4	Factory Pattern (Decks) (Akim)	14
4.5	Strategy Pattern (Sieg) (Akim)	15
4.6	Strategy Pattern (Effekte) Stanislav	16
4.7	Observer Pattern (Lukas)	17
4.8	Gesamt-Klassendiagramm (Akim)	18
		20
5	Implementierung	20
5.2	Initialisierung eines Spiels	21

5.3 Verarbeitung eines Zuges (Stanislav)	22
5.4 Umsetzung der Patterns	22
5.4.1 State Pattern (Lukas).....	22
5.4.2 Command Pattern (Stanislav)	23
5.4.3 Strategy (Stanislav).....	25
5.4.4 Factory (Stanislav)	26
5.4.5 Observer (Lukas)	28
5.5 Showcases (Stanislav)	29
5.5.1 Durak.....	29
5.5.2 Mau-Mau	29
5.5.3 Minigame	29
6 Validierung	30
6.1 Erfüllung der Anforderungen (Akim)	30
6.1.1 Framework (Stakeholder Spielentwickler) (Stanislav)	33
6.1.2 Showcase (Stakeholder User) (Akim)	34
6.1.3 Nicht erfüllte Anforderungen	34
6.1.3.1 Zuleger.....	34
6.1.3.2 Eine offene Angriffskarte	34
6.1.3.3 Ziehreihenfolge beim Nachziehen.....	34
6.1.3.4 Undo bei komplexen Effekten.....	35
6.2 Qualitätssicherungsmaßnahmen	35
6.2.1 Konsistenzanalyse (Iteration 1 → 2)	35
6.2.2 Test-First mit JUnit.....	35
6.2.3 KI-gestützte Architekturanalysen	35
7 Abschlussbetrachtungen	36
7.1 Zusammenfassung (Lukas).....	36
7.2 Ausblick (Lukas)	36
7.3 Fazit (Lukas, Akim).....	37
Anhang 1 Spielregeln für Durak (KI).....	39

Anhang 2 Spielregeln für MauMau (KI)	40
A Verantwortlichkeiten	41
B Abgabe.....	42
C Stundenzettel	43
D Erklärung	46
Literaturverzeichnis	47

1. Einleitung

In dieser Hausarbeit werden die Planung und Umsetzung eines Frameworks zur Erstellung von Kartenspielen behandelt. Um die Funktionsfähigkeit und den Funktionsumfang der entwickelten Engine zu demonstrieren, werden außerdem ein oder mehrere Showcase Beispiele für verschiedene Kartenspielvarianten erstellt. Die durchgeführte Analyse zeigte, dass rundenbasierte Kartenspiele wesentliche gemeinsame Komponenten aufweisen, darunter Enums sowie grundlegende Kartenstrukturen. Zudem sind die Spiele in strukturierte Phasen unterteilt und besitzen eine einheitliche Gesamtmechanik zur Ermittlung des Spielendes. Die Entwicklung jedes Spiels von Grund auf führt jedoch zu Codeduplikaten und Implementierungen, die sich in ihrer Architektur stark voneinander unterscheiden können, was die Möglichkeit ausschließt, verschiedene Spiele auf einer einzigen Engine auszuführen.

Lösung besteht in der Erstellung eines Kerns, der den Spielablauf steuert und verschiedene Muster implementiert, um die Umsetzung verschiedener Varianten von Kartenspielen zu vereinfachen.

1.1 Aufgabenstellung (Akim)

Um dieses Ziel systematisch zu erreichen, umfasst die Vorgehensweise folgende methodische Schritte:

1. Analyse der relevanten Stakeholder sowie Formulierung eindeutiger, lösungsneutraler Ziele und Definition eines fachlichen Glossars.
2. Erstellung von Use-Case-Diagrammen für Framework und Showcase sowie Modellierung des dynamischen Spielablaufs mittels Aktivitätsdiagrammen.
3. Inkrementeller Entwurf der einzelnen Entwurfspatterns (State, Command, Factory, Strategy, Observer) als isolierte Teildiagramme und deren anschließende Zusammenführung in ein einheitliches Gesamt-Pattern-Diagramm.
4. Objektorientierte Implementierung des Klassendiagramms in Java und abschließende Validierung im Rahmen einer Konsistenzanalyse und Qualitätssicherung

1.2 Aufbau der Hausarbeit (Akim)

Der Aufbau der Hausarbeit orientiert sich an der beschriebenen methodischen Vorgehensweise und gliedert sich wie folgt:

- Kapitel 2 - Grundlagen: Vermittelt die verwendeten Technologien, sowie der fünf verwendeten GoF-Patterns.
- Kapitel 3 - Anforderungsanalyse: Dokumentiert die Ergebnisse der Stakeholder-Analyse, die formulierten Ziele, das Glossar, die Use-Case-Diagramme sowie die Aktivitätsdiagramme.

- Kapitel 4 - Modellierung: Beschreibt das Grobdesign und das Feindesign mit allen Teildiagrammen der Patterns sowie das zusammengeführte Gesamt-Pattern-Diagramm.
- Kapitel 5 - Implementierung: Detailliert die Quellcodeumsetzung in Java von framework und showcase.
- Kapitel 6 - Validierung: Überprüft die Implementierung im Rahmen einer Konsistenzanalyse gegenüber den Anforderungen und Use Cases.
- Kapitel 7 - Abschlussbetrachtungen: Führt Zusammenfassung, Ausblick und Fazit auf
- Anhänge - Die Regeln für bestimmte Spiele, Verantwortung und Zeitaufwand für das Projekt

2 Grundlagen

Für ein effizientes und strukturiertes Projektmanagement wurde die Plattform Notion eingesetzt. Hierfür wurde ein schrittweiser Projektplan erstellt, und die anstehenden Aufgaben wurden in Form von Tickets mit Aufwandsschätzungen in Stunden, detaillierten Arbeitsbeschreibungen sowie festgelegten Verantwortlichkeiten strukturiert.

Innerhalb dieser Tickets wurden bei Bedarf Diskussionen geführt, um aufgetretene Fragestellungen zu klären und die Ergebnisse einzelner Teilaufgaben zu dokumentieren. Darüber hinaus diente das Notion-Projekt als zentrale Dokumentationsplattform, in der die ursprüngliche Projektidee sowie die fachlichen Anforderungen gebündelt wurden.

Ergänzend wurden Git und GitHub für die Versionsverwaltung des Quellcodes und der Dokumentation genutzt, um eine kontinuierliche Integration und nachvollziehbare Codeänderungen im Team zu gewährleisten.

2.1 State Pattern (Lukas)

Das State Pattern löst das Problem, dass ein Objekt sich je nach Zustand unterschiedlich verhalten soll. Ohne Muster wird der Zustand als Variable geführt und in jeder betroffenen Methode über Fallunterscheidungen abgefragt. Dadurch verteilen sich die Regeln eines Zustands über viele Methoden und jeder neue Zustand erzwingt Änderungen an allen.

Die Lösung besteht darin, jeden Zustand als eigene Klasse mit gemeinsamem Interface zu modellieren. Der Kontext hält nur eine Referenz auf den aktuellen Zustand und delegiert die zustandsabhängigen Aufrufe an ihn. Ein Zustandswechsel ist damit ein Austausch des referenzierten Objekts, und neue Zustände lassen sich ergänzen, ohne bestehende zu verändern.

Für den Übergang bestehen zwei Varianten: Entweder kennt der Zustand den Kontext und setzt den Nachfolger dort selbst, oder die zustandsverändernde Methode gibt den Folgezustand zurück und der Kontext übernimmt ihn. In dieser Arbeit wird die zweite Variante nach [KleukFolien, F. 308 ff.] gewählt, da sie ohne wechselseitige Referenzen zwischen Kontext und Zustand auskommt.

2.2 Factory Pattern (Akim)

Das Factory Pattern dient zur Kapselung und Systematisierung der Objekterzeugung. Es stellt sicher, dass Objekte stets in einer definierten, korrekten Form erzeugt werden und die nutzenden Klassen vom konkreten Erstellungsprozess entkoppelt bleiben.

Das Muster kommt zum Einsatz, wenn im System viele verschiedene, aber gleichartige Objekte, wie z.B. unterschiedliche Kartendecks, verwaltet werden sollen. Eine zentrale Steuerungsklasse muss diese Objekte zwar bearbeiten können, benötigt dafür aber nur deren gemeinsame Schnittstelle. Dazu werden zwei abstrakte Ebenen eingeführt, eine für die Objekterzeugung und eine für den Objektzugriff. Der Aufrufer kennt dabei nur diese abstrakten

Ebenen und bleibt unabhängig vom konkreten Typ des erzeugten Exemplars. [Kle26, F. 387-388 ff.]

2.3 Command Pattern (Akim)

Das Command Pattern eignet sich hervorragend für die Kapselung von Steuerprozessen und ermöglicht es, ausgeführte Actions im System wieder rückgängig zu machen. Voraussetzung hierfür ist, dass alle durch die Action bewirkten Änderungen bekannt und vollständig reversibel sind. Für die Umsetzung von Undofunktionalität stehen folgende Varianten zu Verfügung:

1. Jedes Command Objekt besitzt eine eigene `undo()` Methode und wird nach der Ausführung in einer Collection gespeichert.
2. Die `execute()` Methode gibt ein eigenes Undo-Command-Objekt als Ergebnis zurück.
3. Undo- und Command Objekte besitzen keine gemeinsame Klasse oder Schnittstelle

Im Rahmen dieser Arbeit wird Ansatz 1 gewählt. Dabei implementiert jede konkrete Command-Klasse im Framework neben der `execute()` Methode auch eine `undo()` Methode. Ausgeführte Commands werden in einer `CommandHistory` gespeichert, was eine direkte und schrittweise Rücknahme von Spielzügen ermöglicht. [KleukFolien S. 325-328ff]

2.4 Observer Pattern (Lukas)

Um eine grafische Benutzer Oberfläche anzubieten die flüssig durchläuft muss der Benutzer durch seine Aktionen entscheiden was wann ausgeführt wird, so ist eine unendliche `gameLoop` (wie im ersten Modellierungsschritt später näher gezeigt) eher nicht zu verwenden und ein Beobachter muss ein Subjekt beobachten und über Änderungen informiert werden, um dann die notwendigen Aktionen auszuführen. So meldet sich ein Beobachter bei einem Subjekt in diesem Fall der Controller Klasse `Game` an und wird über Änderungen des Zustands informiert. [KleukFolien S. 276ff]

2.5 Strategy Pattern (Akim)

Das Strategy Pattern dient dazu, ein bestimmtes Verhalten von seiner Verwendung zu entkoppeln, indem das zugehörige Verfahren in ein eigenes Objekt gekapselt wird, das ein einheitliches Interface implementiert. Auf diese Weise können für dieselbe Aufgabe mehrere alternative Implementierungen bereitgestellt werden, die zur Laufzeit beliebig ausgetauscht werden können, ohne dass der aufrufende Code Änderungen erfahren muss. In dieser Arbeit wird das Muster in zwei Bereichen verwendet.

Erstens zur Bestimmung der Spielende Bedingungen, indem ein abstraktes Interface zur Prüfung von Sieg- bzw. Verlustbedingungen definiert wird und je nach Spielvariante die passende Strategieimplementierung eingesetzt wird.

Zweitens werden Spieleffekte über das Strategy-Pattern realisiert. Die Herausforderung besteht darin, unterschiedliche Effekte flexibel einzusetzen, ohne die allgemeine Spiellogik für jeden Effekt anpassen zu müssen. Durch die Kapselung der einzelnen Verhaltensweisen können

Effekte unabhängig voneinander ausgetauscht oder erweitert werden, wodurch das System flexibel und leicht erweiterbar bleibt. [KleukFolien S. 306-308ff]

3 Anforderungsanalyse

Die Anforderungsanalyse stellt die Grundlage der Systementwicklung dar. Um ein gemeinsames Verständnis aller Begriffe und Ziele zu gewährleisten, wurde die Analyse in folgenden 5 systematischen Schritten durchgeführt

3.1 Glossar (Akim)

Der erste Schritt war die Erstellung eines Glossars. Dies diente dazu, einheitliche Begriffe für das gesamte Projekt festzulegen, die sowohl in der Planung als auch bei der Implementierung im Code verwendet werden. Das Glossar wurde im root des Doc-Verzeichnis des Projekts als Markdown Datei abgelegt, worauf jederzeit zugegriffen werden kann.

Begriff	Beschreibung
GameLoop	zentrale Kontrollmechanismus des Frameworks, der die Phasen des Spiels sequenziell durchläuft
Card	Ein abstraktes Element im Spiel. Jede Karte besitzt spezifische Attribute (z. B. Wert, Farbe)
Deck	geordnete oder ungeordnete Sammlung von Card-Objekten. Das Framework stellt Fabrikmethoden zur Generierung verschiedener Decks bereit.
Hand	Sammlung aller Karten die sich im besitz des Spielers befinden
Table	Tisch bezeichnet alle für die Spieler sichtbaren Karten
Phase	Definierter Abschnitt innerhalb eines Spielzugs (z. B. Ziehphase, Spielphase, Schlagphase, Dazulegenphase, Prüfphase). Die Zustände werden durch das State Pattern gesteuert.
Command	Ein Objekt, das eine konkrete Spielaktion (z. B. "Karte ausspielen") kapselt, um diese auszuführen oder rückgängig zu machen (Undo).
Win Condition	Bezeichnet eine Kondition die unter Erfüllung den Spieler gewinnen lässt und immer wieder vom System geprüft wird. Bspw. keine Handkarten mehr

Tabelle 1: Auszug des fachlichen Glossars

3.2 Stakeholder und Ziele (Akim)

Anschließend wurden die zentralen Stakeholder sowie deren Ziele definiert. Wie in Abbildung 3.1 dargestellt, lassen sich zwei primäre Hauptakteure festlegen. Der Entwickler, der mit dem fertigen Framework weitere Anwendungen entwickelt, sowie der User, der die Showcase Anwendung nutzt.

Abbildung 3.2 verdeutlicht die Gegenüberstellung dieser Akteure und ihrer jeweiligen Anforderungen. Das Hauptziel auf Framework Ebene besteht in der Bereitstellung einer

flexiblen Architektur, die das Entwerfen unterschiedlicher Kartenspiele ermöglicht. Demgegenüber steht das Ziel des Users, das Framework anhand eines funktionierenden Showcase-Spiels zu validieren. Diese Zielstruktur bildet die direkte Grundlage für die nachfolgende Use-Case Analyse. Hier sind noch solche Namen wie GameLoop und GameState zu sehen, diese werden noch in diesem Kapitel weiter zu sehen, da diese erst zu einem späteren Zeitpunkt umbenannt und angepasst worden sind, was man weiter in dem Bericht sieht.

Ziel	1. Den Programmierern eine flexible Architektur zur Erstellung beliebiger Kartenspiele zur Verfügung stellen, mit der Möglichkeit, den GameLoop und den GameState einfach zu steuern
Stakeholder	Spielentwickler
Auswirkung auf Stakeholder	Die Spielentwickler erhalten einen Framework, das die Implementierung verschiedener spielspezifischer Aktionen und variabler Winconditions ermöglicht. Das System unterstützt standardmäßig die Generierung und den Einsatz von unterschiedlichen Kartendecks (z. B. bestehend aus 32 oder 52 Card-Objekten)
Randbedingungen	Das System soll die zukünftig mögliche erweiterungen für weitere Kartendecks unterschützen
Abhängigkeiten	-
Sonstiges	-

Abbildung 3.2.1: Ziel 1, Stakeholder: Spielentwickler

Ziel	2. Validierung und Demonstration des Frameworks durch die prototypische Implementierung eines konkreten Kartenspiels (Showcase)
Stakeholder	Player(User, Computer)
Auswirkung auf Stakeholder	Player: Erhält ein vertiges Produkt (Kartenspiel), mit klarem Spielablauf und allen Notwendigen funktionen.
Randbedingungen	Da es nur ein Showcase ist muss die UI nicht kompliziert sein und darf starkt vereinfacht und mit KI generiert werden
Abhängigkeiten	Basiert komplett auf dem Ziel1, weil das Spiel auf dem Framework aufgebaut ist
Sonstiges	-

Abbildung 3.2.2: Ziel 2, Stakeholder: User

3.3 Use-Case-Diagramm (Akim)

Für das in Abbildung 3.3 dargestellte Use-Case-Diagramm bildeten die im vorherigen Schritt erarbeiteten Akteure die Grundlage. Der Fokus lag hierbei primär auf der Beschreibung des Showcase Ablaufs und weniger auf dem Framework.

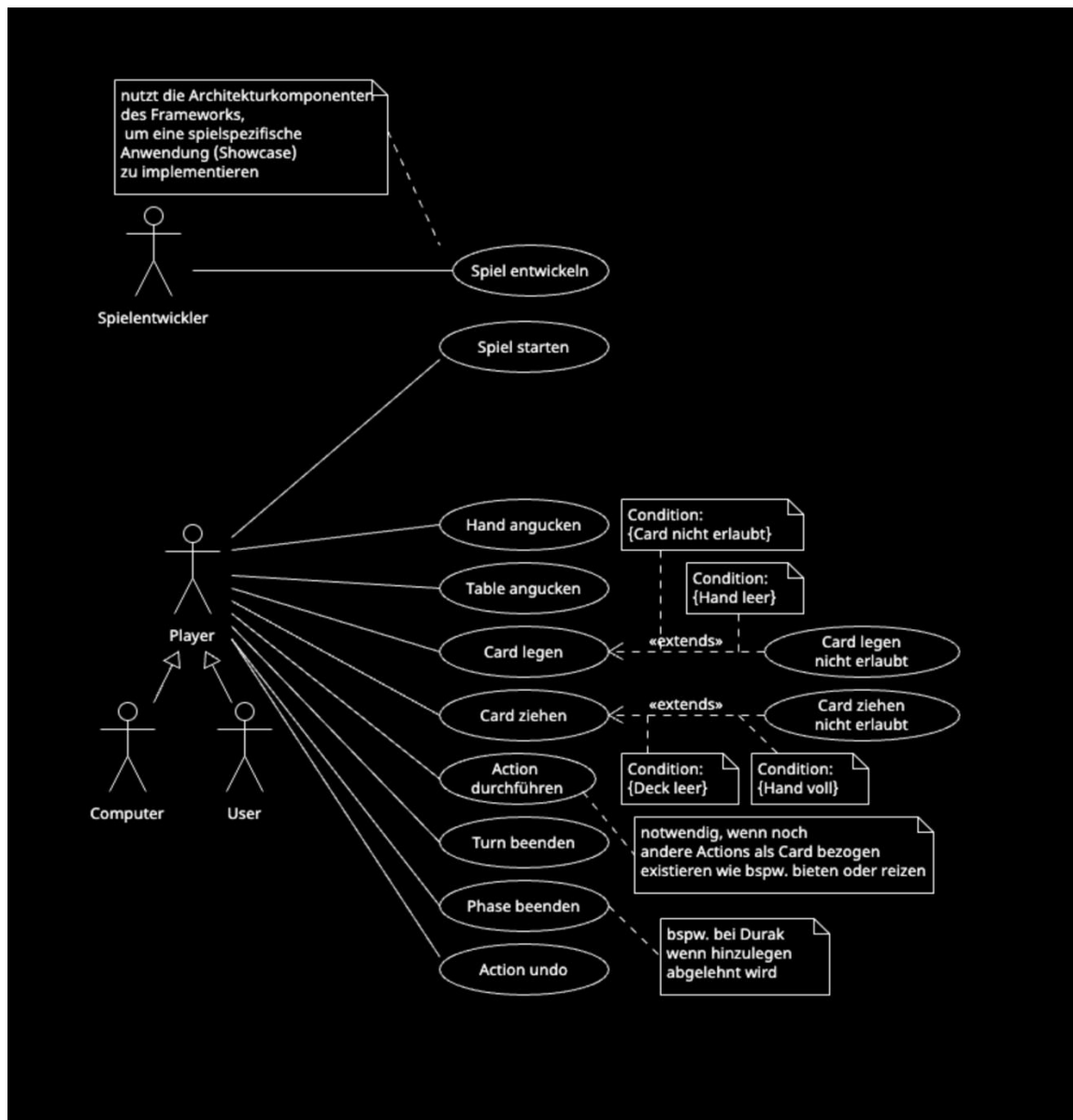


Abbildung 3.3: Use-Case

3.4 Use-Case-Dokumentation (Akim)

Basierend auf dem Use-Case-Diagramm wurde eine detaillierte Use-Case-Dokumentation ausgearbeitet, wie sie beispielhaft in Abbildung 3.4 dargestellt ist. Diese Dokumentation fasst die wesentlichen Systemabläufe, Vor- und Nachbedingungen sowie Ausnahmeszenarien übersichtlich zusammen. Auf dieser Grundlage wird der genaue Spielablauf präzisiert und die Überführung in das Aktivitätsdiagramm ermöglicht.

Name des Use Case	[2]	Showcase: Anwendung
Nummer	[1]	U1
Paket	[1]	-
Erstellung	[1]	Akim Zdorovtsov
Version	[1]	1.1, 18.07.2026, System-Aktor bereinigt und Ablauf verfeinert
Kurzbeschreibung	[1]	Ein Player initiiert den Start einer neuen Spielpartie, woraufhin die Engine den initialen GameState aufbaut, das Deck mischt und automatisch die Cards in der Hand verteilt. Woraufhin folgt der klassischer GameLoop Ablauf
beteiligte Akteure	[2]	Player (User / Computer), Entwickler
Fachverantwortlich	[1]	noch nicht gesetzt
Referenzen	[2]	Projektspezifisches Glossar V1.0
Vorbedingungen	[2]	Die Engine wurde durch den Spielentwickler fehlerfrei initialisiert. Ein vollständiges Deck (32 oder 52 Cards) ist im Speicher vorhanden. Alle wichtige Actions sind erweitert und liegen korrekt da.
Nachbedingungen	[2]	Ein konsistenter GameState ist etabliert. Das Deck ist erfolgreich gemischt, und Objekte Hand bei Player sind mit Objekten Card befüllt.
typischer Ablauf	[2]	1. Der Player wählt über die Benutzungsschnittstelle die Option zum Starten des Spiels. 2. Die Engine mischt das zugewiesene Deck selbstständig im Hintergrund. 3. Die Engine generiert und verteilt automatisch Card-Objekte in die Hand jedes Players. 4. Die Engine startet den GameLoop und öffnet die erste Phase des ersten Turns. 5. Der Player wählt eine Card aus seiner Hand aus. 6. Die Engine überprüft die Gültigkeit der Card. 7. Die Card wird auf den Table gelegt. 8. Die Action wird in einen Command verpackt und dem Spielverlauf hinzugefügt.
alternative Abläufe	[2]	Card ungültig: Die Engine zeigt eine Fehlermeldung an und fordert zu einer weiteren Action auf. Zug rückgängig machen: Der Player ruft Undo auf.
Kritikalität	[3?]	Sehr hoch, da dieser Ablauf die fundamentale Voraussetzung für eine funktionale Anwendung ist

Abbildung 3.4: Use-Case Dokumentation

3.5 Rupp Schablone (Lukas)

Im Rahmen der Anforderungsanalyse wurde aus dem Use Case Diagramm folgende Anforderungen gemäß der Rupp-Schablone abgeleitet.

Game Init

1. Nach dem das System gestartet wurde soll das System ein Deck anlegen
2. Nach dem Anlegen des Decks soll das System dieses mischen
3. Nach dem das Deck gemischt wurde soll das System die Hand austeilen
4. Sobald alle Player eine Hand haben, soll das System die Reihenfolge der Player bestimmen.

Game Loop

5. In jeder Round vor dem Turn des ersten Players soll das System die Möglichkeit haben den Table zu aktualisieren
6. Nach dem der Table und Hand vorbereitet sind soll das System dem aktiven Player die Möglichkeit bieten seinen Turn durchzuführen

Turn Loop

7. Wenn ein Turn begonnen wird, soll das System dem Player die Möglichkeit bieten die Phasen zu durchlaufen

Phase Loop

8. Wenn eine Phase begonnen wird, soll das System dem Player die Möglichkeit bieten Actions durchzuführen

Action Loop

9. Wenn eine Action durchgeführt wird, soll das System prüfen, ob die Action gültig ist
10. Ist die Action ungültig so soll das System dem Player die Möglichkeit bieten eine andere Action durchzuführen
11. Ist die Action gültig so soll das System prüfen, ob eine weitere Aktion möglich ist
12. Ist eine weitere Aktion möglich so soll das System dem Player die Möglichkeit bieten eine weitere Action durchzuführen

13. Kann der Player eine weitere Aktion durchführen soll das System dem Player die Möglichkeit bieten das Abzulehnen und in die nächste Phase zu gehen
14. Ist die Action gültig so soll das System dem User (menschlicher Player) die Möglichkeit bieten ein Undo (Action rückgängig) zu machen
15. Versucht der User ein Undo zu machen prüft das System, ob ein Undo möglich ist
16. Ist ein Undo möglich so soll das System dem User die Möglichkeit bieten eine neue Action durchzuführen
17. Ist ein Undo nicht möglich so soll das System dem User das anzeigen und die Möglichkeit bieten eine weitere Aktion durchzuführen
18. Am Ende einer Phase soll das System prüfen, ob eine Win Condition erfüllt ist.
19. Ist eine Win Condition erfüllt soll das System den Winner anzeigen
20. Nach dem Prüfen einer Win Condition soll das System die nächste Phase einleiten
21. Ist keine nächste Phase einleitbar (letzte Phase) soll das System den Turn des Players beenden
22. Nach beenden des Turns soll das System prüfen, ob ein nächster Player seinen Turn beginnen darf
23. Darf kein nächster Player seinen Turn beginnen so soll das System die nächste Round starten

3.6 Aktivitätsdiagramm (Lukas)

Abschließend zur Anforderungsanalyse bildet das in Abbildung 3.6 dargestellte Aktivitätsdiagramm. Dieses modelliert detailliert den dynamischen Spielablauf sowie die verschachtelte Struktur der einzelnen Spielschleifen wie GameLoop, TurnLoop, PhaseLoop und ActionLoop inklusive der Aktionsvalidierung und der Undofunktionalität. Damit stellt das Aktivitätsdiagramm den zentralen Orientierungspunkt für die nachfolgende Architektur- und Modellierungsphase des Frameworks dar.

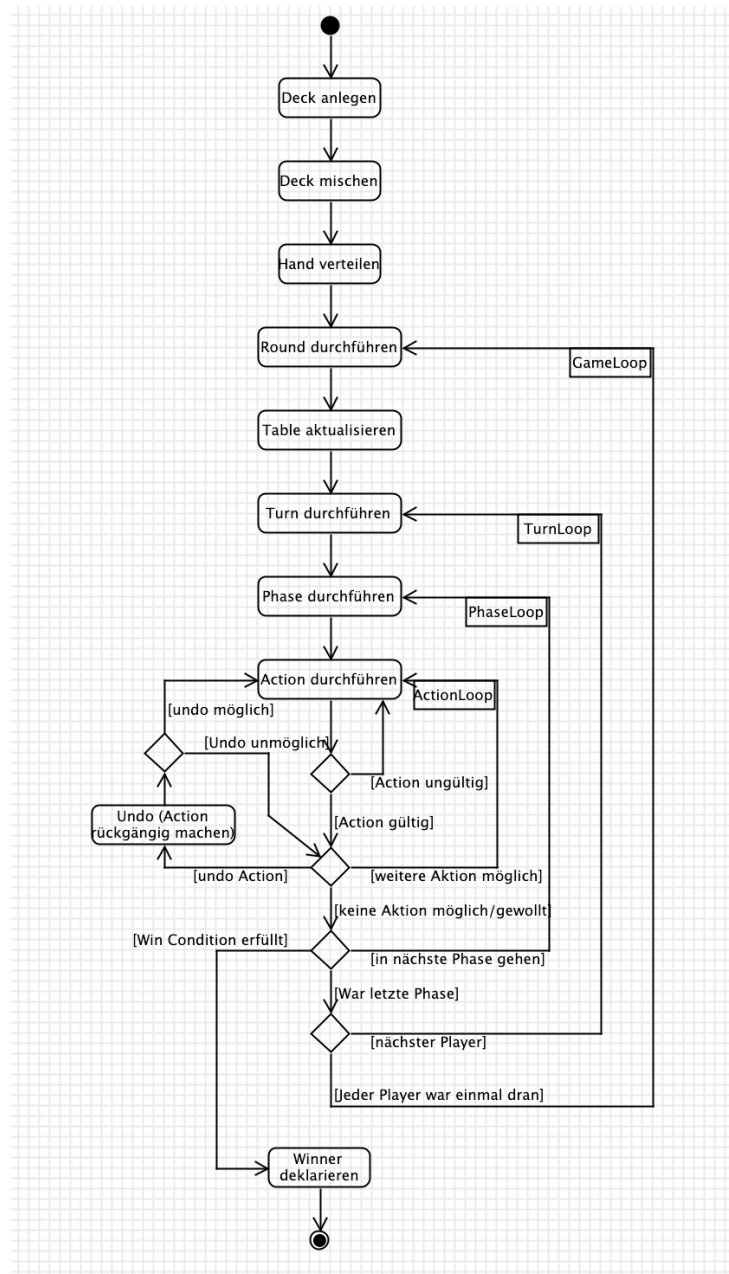


Abbildung 3.6: Aktivitätsdiagramm

4 Modellierung

Im Folgenden soll die Modellierung des Projekts mit in Form der Klassendiagrammen erfolgen wobei die Pattern erstmal einzeln betrachtet und anschließend in einen Gesamtkontext übernommen werden.

4.1 Grundarchitektur des Frameworks (Stanislav)

Die Grundarchitektur des Frameworks besteht aus allgemeinen Modellklassen, die unabhängig von einem konkreten Kartenspiel verwendet werden können. Die Klasse Game verwaltet den zentralen Spielzustand und reguliert Spieler, Deck, Tisch, Hand und Spielphasen. Karten werden durch Card dargestellt und mithilfe von CardCollection in Händen, Decks oder auf dem Tisch verwaltet. Über das Interface GameSetup können verschiedene Spiele unterschiedlich

initialisiert werden. Abstraktion und Interface ermöglichen es, neue Kartenspiele zu ergänzen, ohne die Grundstruktur des Frameworks wesentlich zu verändern.

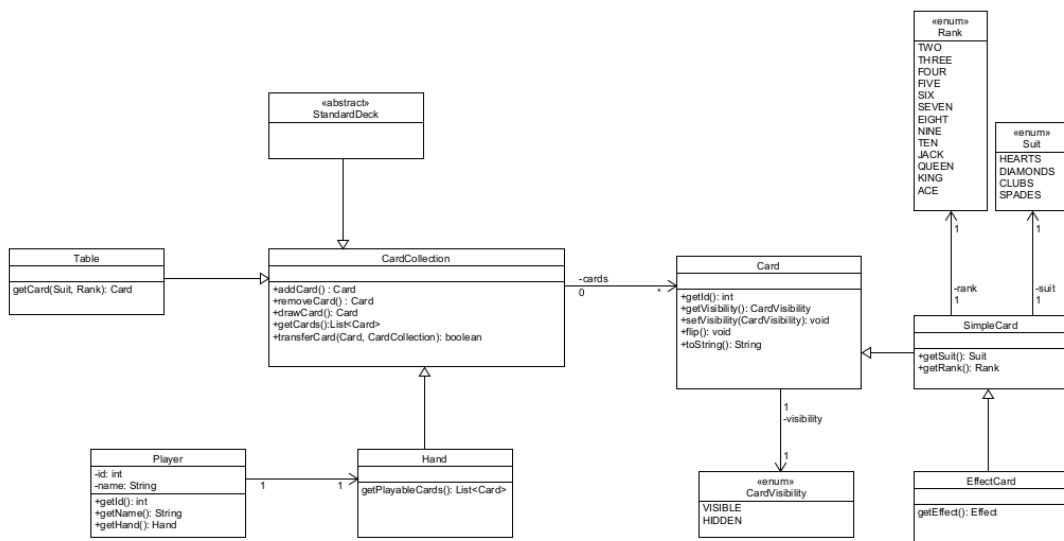


Abbildung 4.1: Basis Klassendiagramm

Oben vorgestellte Version wurde bei der Implementierung vereinfacht s. 5. Implementierung

In der ersten Version wurden Hand und Table als eigene Klassen modelliert, obwohl beide hauptsächlich eine Sammlung von Karten darstellen. Zusätzlich wurde zwischen SimpleCard und EffectCard unterschieden. Player besaß eine Hand, während Table die ausgespielten Karten verwaltete. Diese Struktur war stärker spezialisiert, führte jedoch zu mehreren Klassen mit ähnlichen Aufgaben.

4.2 State Pattern (Phasen) (Lukas)

Um die verschiedenen Phasen der Kartenspiele mit jeweils passenden Attributen zu versehen, wurde das State Pattern gewählt [KleukFolien S.308] (Variante nach Folie 308). Das Framework soll ein Interface Phase anbieten welches von den verschiedenen Spielen und ihren individuellen Phasen spezialisiert werden soll. Dafür hält die Controller Klasse Game die derzeitige Phase und Phase gibt die implementierung einer Methode zur Validierung der Aktion/Commands vor und die implementierung einer Funktion zur Festlegung der Folgephase. Vergleiche Abbildung 4.2.

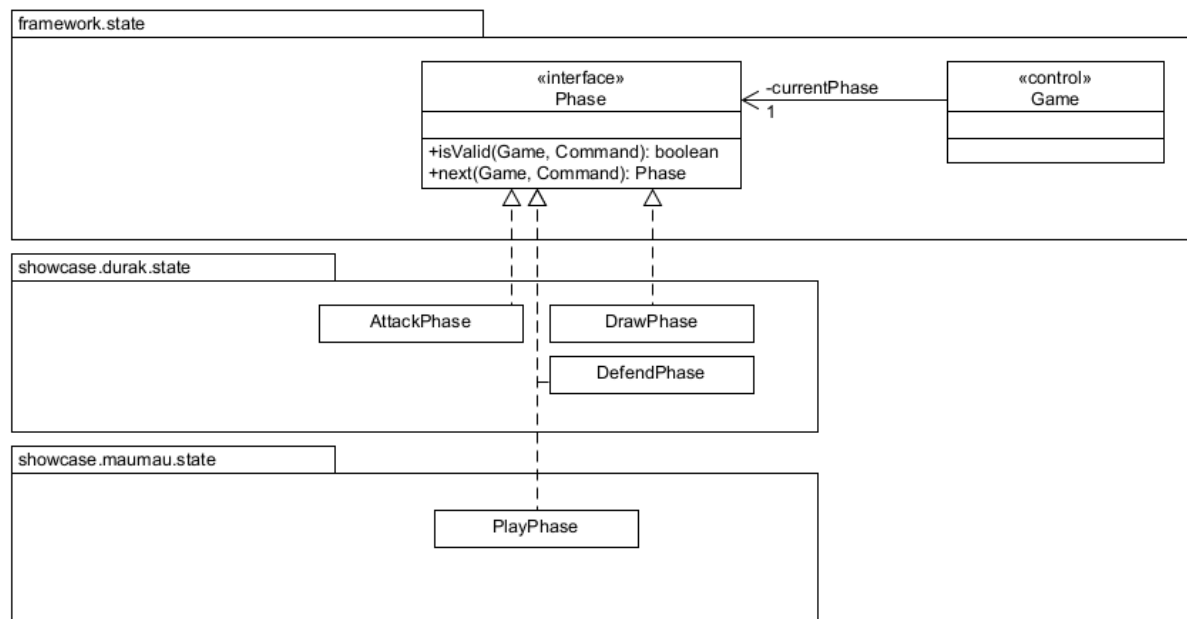


Abbildung 4.2: State Pattern

4.3 Command Pattern (Undo) (Akim)

Für die Umsetzung der Action wurde das Command Pattern gewählt. Dieser Ansatz ermöglicht es, jeden Spielzug als eigenständiges Objekt zu kapseln, was sowohl eine universelle Umdomöglichkeit als auch die Erstellung spielspezifischer Befehle in den Showcase Anwendungen unterstützt.

Wie in Abbildung 4.3 dargestellt, bildet die Interface Command den zentralen Vertrag, der durch konkrete Actions in den Showcasespaketen, z. B. AttackCardCommand bei Durak oder BetCommand bei Poker, implementiert wird. Die gemeinsame abstrakte Klasse AbstractCommand speichert dabei die Referenz auf den ausführenden Player, was für alle ererbenden Klassen ein gemeinsames Merkmal ist.

Zur Umsetzung der Undo-Funktionalität wurde die Klasse CommandHistory konzipiert. Sie nimmt die Rolle des Invokers ein und verwaltet alle ausgeführten Befehle in einem Stack<Command>. Dieser arbeitet nach dem LIFO-Prinzip, was die exakte Rücknahme der jeweils letzten Aktionen ermöglicht. Zudem erlaubt diese Historie die Ausgabe des Spielverlaufs in der Benutzeroberfläche. Die Klasse CommandHistory wird dann als Objektinstanz direkt in den zentralen Controller - Game eingebunden.

Command ist das Command-Interface. RaiseCommand, CallCommand, FoldCommand, BetCommand, CheckCommand, DrawCardCommand, AttackCardCommand, TakeCardCommand und ThrowInCardCommand sind Concrete Commands. Player, Pot, Deck, Table und Card sind die Receiver. CommandHistory ist der Invoker.

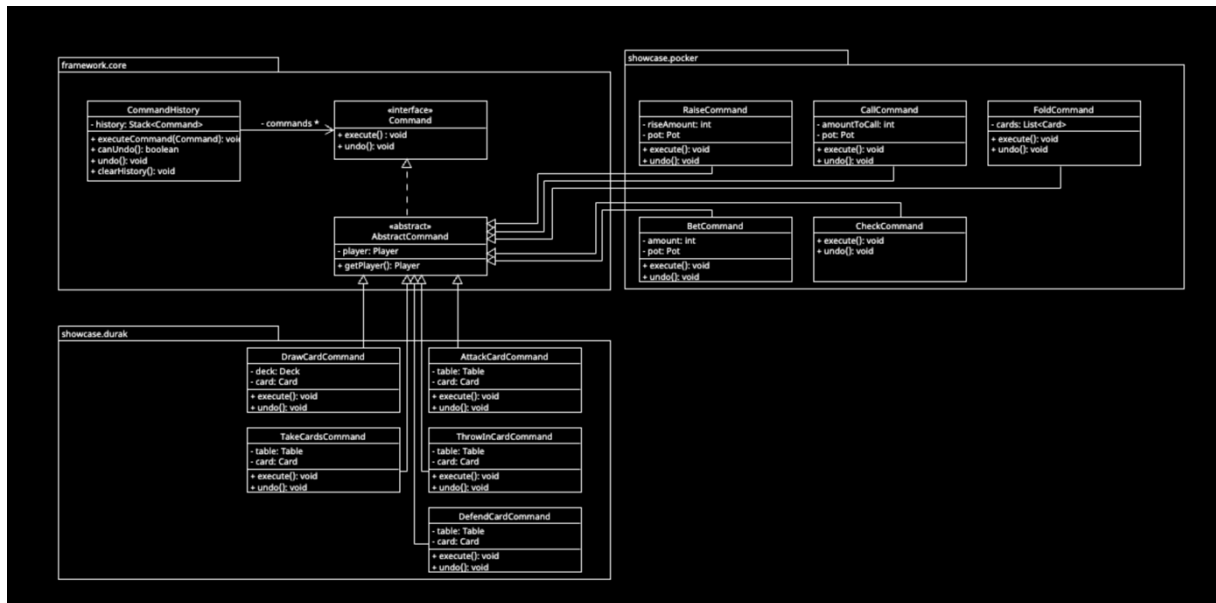


Abbildung 4.3: Undo Command Pattern UML

4.4 Factory Pattern (Decks) (Akim)

Für die Erzeugung von Kartendecks erwies sich die Anwendung des Factory Method Patterns als optimale Lösung. Dieser Ansatz ermöglicht es, spielspezifische Decks mit variierender Kartenanzahl zu erstellen, während gemeinsame Methoden und Strukturen im Framework wiederverwendet werden und gleichzeitig Raum für spezifische Eigenschaften verbleibt.

Wie in Abbildung 4.4 dargestellt, erhält jede Showcase Anwendung eine eigene konkrete Fabrik, welche das Deck für das jeweilige Spiel erzeugt. Dies erlaubt es beispielsweise, für den Durak Showcase ein Deck mit 36 Karten sowie einer festgelegten TrumpfSuit zu erstellen, während für andere Spielvarianten Decks mit 32 oder 52 Karten generiert werden können.

Hierbei nimmt Deck die Rolle einer reinen Interface ein. Eine denkbare Alternative hätte darin bestanden, Deck als abstrakte Klasse zu gestalten, welche von CardCollection erbt. Dies hätte jedoch zu einer tiefen, mehrstufigen Vererbungshierarchie geführt. Die Entscheidung für ein Interface bietet an dieser Stelle eine deutlich höhere Flexibilität und gewährleistet eine lose Kopplung zwischen den Komponenten des Frameworks. Eine vertiefte Betrachtung dieser Zusammenhänge erfolgt in Kapitel 4.5 Gesamt-Klassendiagramm.

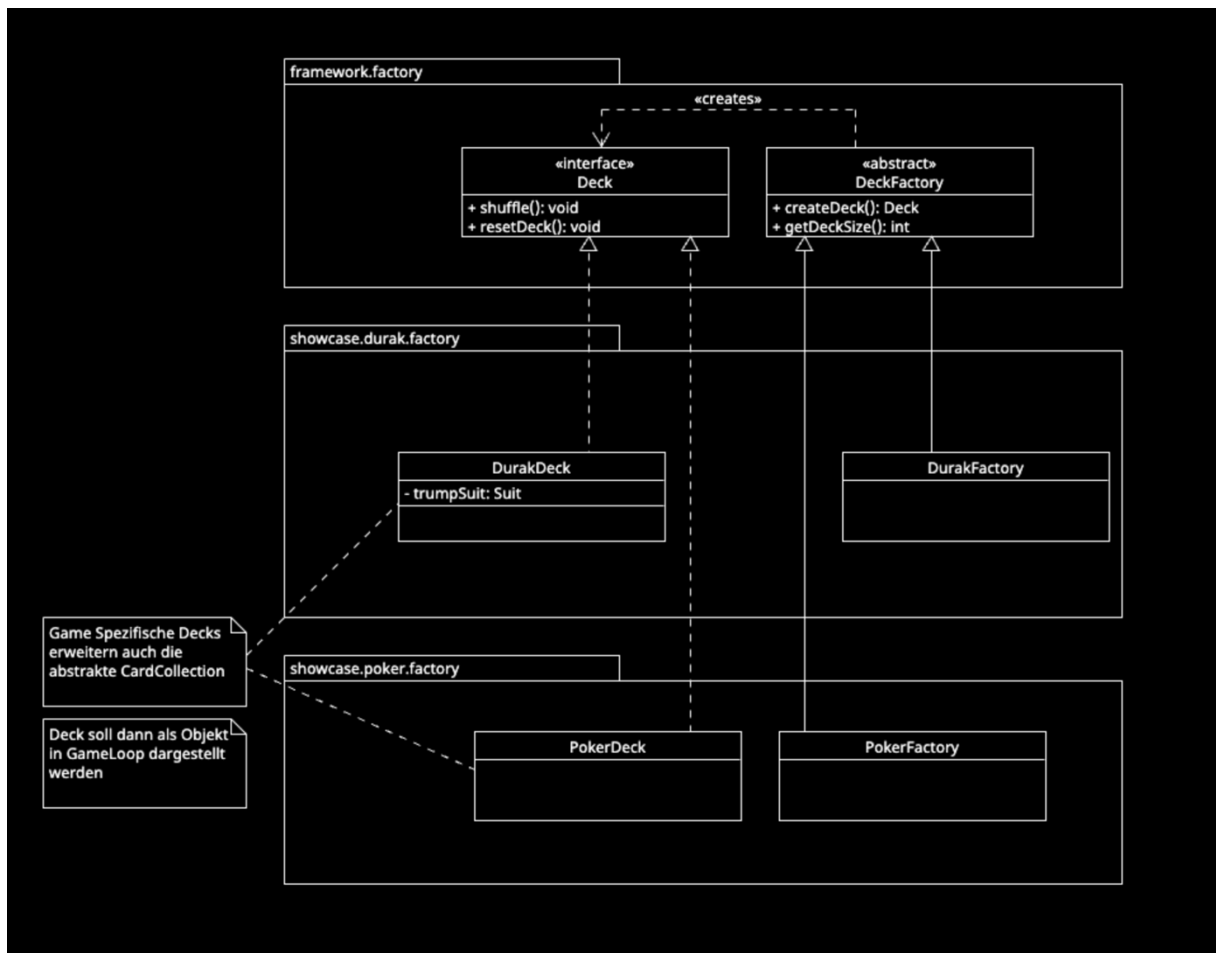


Abbildung 4.4: Factory Pattern

4.5 Strategy Pattern (Sieg) (Akim)

Für die Ermittlung von Spielende und Siegbedingungen erweist sich die Anwendung des Strategy Pattern als optimale architektonische Lösung. Dieser Ansatz ermöglicht es, die Regelprüfung in eingeständige Strategieklassen auszulagern, indem ein Interface `WinCondition`, in den jeweiligen Showcase Anwendungen spezifisch implementiert wird.

Wie in der Abbildung 4.5 dargestellt, definiert die Schnittstelle `WinCondition` die Methoden `isGameOver(GameLoop)` und `getWinner(GameLoop)`, die Übergabe von `GameLoop`-Objekt als Parameter gewährt den vollen Zugriff auf den aktuellen Spielzustand, ohne die Kapselung der Komponenten zu verletzen. Der Einsatz des Strategy Patterns ist essenziell, da z.B. während `DurakWinCondition` prüft, ob nur noch ein Spieler Karten hält der Verlierer, gewinnt bei `MauMauWinCondition` der Spieler ohne Handkarten. Der Controller `GameLoop` hält die Referenz auf die Strategie und führt die Prüfung nach jedem Zug aus.

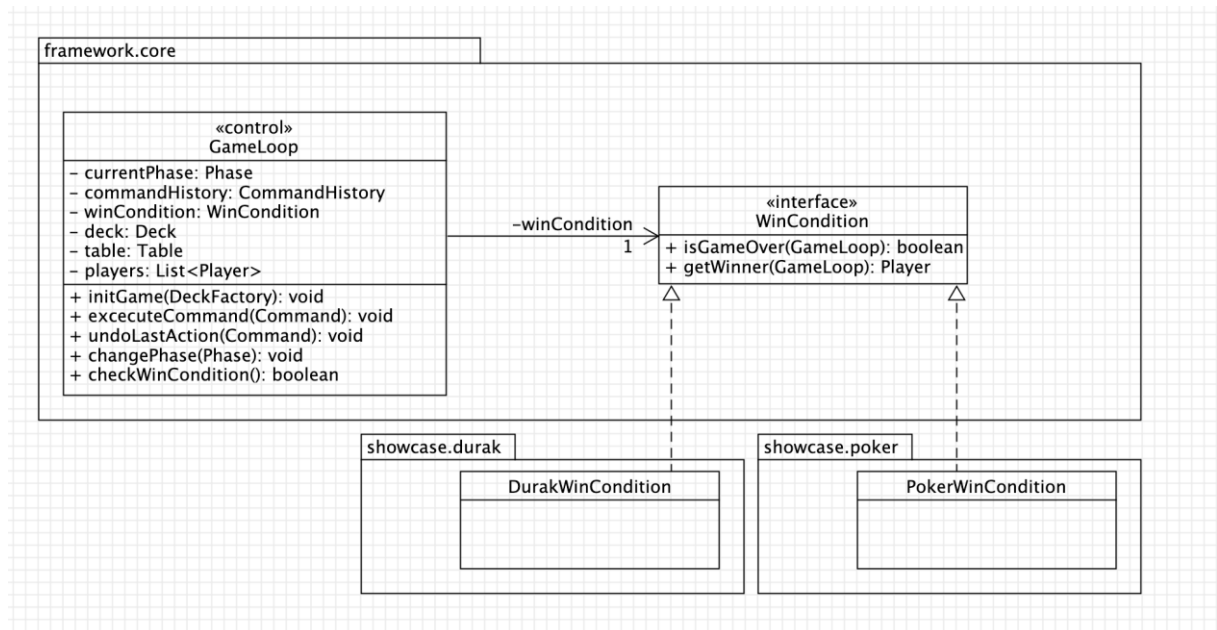


Abbildung 4.5: Strategy Pattern

4.6 Strategy Pattern (Effekte) Stanislav

Viele Kartenspiele besitzen nicht nur die Eigenschaften Suit und Rank einer Karte. Einige Karten haben zusätzlich besondere Effekte. Beispiele dafür sind Mau-Mau, Uno oder ähnliche Kartenspiele. Eine Karte kann dazu führen, dass der nächste Spieler aussetzt, zusätzliche Karten ziehen muss oder eine neue Farbe gewählt werden darf. Ohne ein Entwurfsmuster würden diese Sonderregeln an vielen Stellen stehen, wo das Spiellogik, Phasen oder Commands implementiert wurden. Es ist keine gute Idee, all diese mithilfe endloser if-else-Konstruktionen fest zu verdrahten. Dadurch wird der Code schwer zu verstehen, zu testen und zu erweitern.

Für dieses Problem wurde das Strategy Pattern gewählt. Dieses Muster kapselt jede Regel als eigene Strategie. Alle Strategien besitzen über das Interface “Effect” dieselbe Operation `apply(Game)`, implementieren diese jedoch unterschiedlich.

Eine `EffectCard` erhält beim Erzeugen des Decks eine konkrete Strategie beispielsweise `DrawTwoEffect`, `SkipEffect` oder `ChooseSuitEffect`. Die Strategie wird dabei über den Konstruktor übergeben. Beim Auspielen von einer Karte wird die Methode `apply(Game)` über das Interface “Effect” aufgerufen. Erst zur Laufzeit wird anhand des tatsächlich gesprochenen Objekts entschieden, welche konkrete Implementierung ausgeführt wird. Dadurch verwendet die Implementierung dynamischen Polymorphismus. Nach der Ergänzung der Karte wird ihre Strategie nicht mehr ausgetauscht. Ein solcher Austausch zur Laufzeit wäre technisch möglich, ist für die vorliegende Anwendung jedoch nicht erforderlich, da die Effekte der Karten fast immer durch die Spielregeln festgelegt sind. Abbildung: Die Darstellung in Abbildung 4.6 orientiert sich am Lösungsbeispiel für das Strategy Pattern aus den Vorlesungsfolien. [KleukFolien S.307]

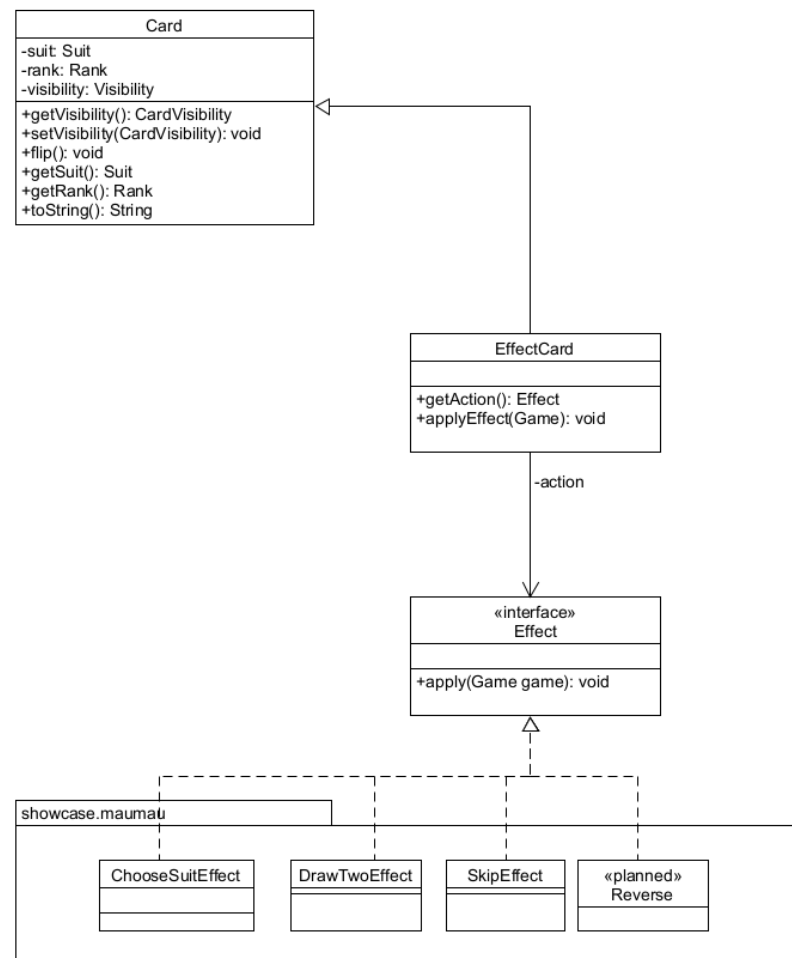


Abbildung 4.6: Strategy Pattern (Effect)

4.7 Observer Pattern (Lukas)

Nach dem Umbau von Konsolen Anwendung mit `gameLoop()` auf GUI wurde es notwendig sich von der statische Ereignisschleife zu trennen und auf ein Observer-Pattern umzubauen. Da dieses zusätzliche Pattern im ersten Modellierungsiteration nicht geplant gewesen ist, ist dieses Pattern in den ersten UML-Diagrammen nicht berücksichtigt.

Rollenzuweisung nach Folie S.277 ist in der Tabelle 4.7 gegenübergestellt [KleukFolien S.277]

Vorlage aus Folien	Im Projekt umgesetzt
Subjekt / KonkretesSubjekt	Game
anmelden(Beobachter)	Game.addGameListener(GameListener)
informieren()	notifyStateChanged() / notifyGameOver() / notifyInvalidMove()
Beobachter	GameListener

aktualisieren()	onStateChanged(Game) / onGameOver(Player) / onInvalidMove(Command)
KonkreterBeobachter	Showcase Controller Klassen

Tabelle 4.7: Strategy Pattern (Effect)

Für die konkrete Implementierung in diesem Projekt wurde sich für drei verschiedene Callbacks entschieden damit die GUI unterschiedlich reagieren kann

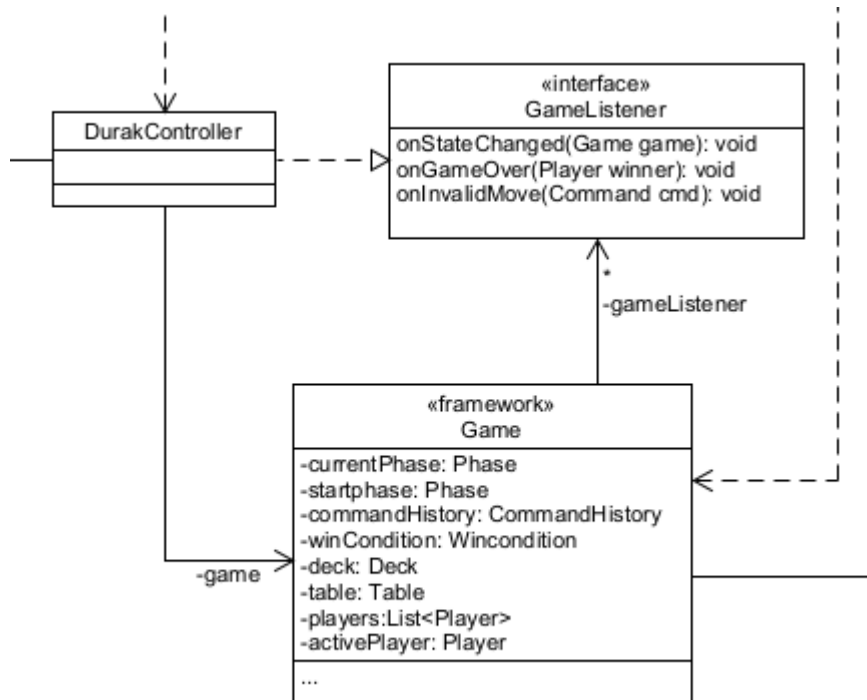


Abbildung 4.7: Auszug Observer Pattern

4.8 Gesamt-Klassendiagramm (Akim)

Den konzeptionellen Abschluss der Modellierung bildet das in Abbildung 4.8 dargestellte Gesamt-Klassendiagramm, welches alle zuvor beschriebenen Entwurfsmuster in einem harmonischen Gesamtsystem vereint. Bei dieser Zusammenführung wurden gezielte Architekturkorrekturen vorgenommen.

Das in Abschnitt 4.4 beschriebene Factory Method Pattern wurde durch die Einführung der abstrakten Klasse StandardDeck optimiert. Da Java keine Mehrfachvererbung von Klassen unterstützt, konnte das Interface Deck nicht direkt von der abstrakten Klasse CardCollection erben. Eine Ausführung beider Konzepte als reine abstrakte Klassen hätte in den Showcase Anwendungen zu unzulässigen Vererbungskonflikten geführt.

Durch die Struktur *StandardDeck extends CardCollection implements Deck* wird eine saubere Trennung erreicht. Das Interface Deck garantiert die lose Kopplung für die DeckFactory, während StandardDeck die wiederverwendbaren Kartenfunktionen aus CardCollection

bereitstellt. Jedes konkrete Showcasedeck muss dadurch lediglich von StandardDeck erben, was eine einfache Vererbungslinie wahrt, und Codeduplizierung verhindert.

Im Zentrum dieser integrierten Architektur steht die Klasse Game, als zentraler Controller der Zustandsverwaltung. Diese Klasse wurde im Zuge des Refactorings von GameLoop in Game umbenannt, da der Name Game die umfassende Rolle der Steuerung und Zustandsverwaltung präziser wiedergibt. Sie führt die einzelnen Entwurfsmuster zusammen, indem sie die aktuelle Phase speichert, die CommandHistory zur Ausführung und Rücknahme von Zügen verwaltet, das geladene Deck einbindet und nach jedem Zug die Schnittstelle WinCondition zur Ermittlung des Spielendes aufruft und zusätzlich das ganze Spiel initialisiert.

Ergänzt wird dieses System durch die grundlegende Struktur. Jeder Player verfügt über eine Hand, während der Spieltisch über eine Table Objekt repräsentiert wird, welche beide ihre Basisfunktionalitäten von CardCollection erben. Das Gesamt-Klassendiagramm verdeutlicht somit die Entkopplung zwischen dem generischen Framework und den konkreten Showcase Anwendungen, wodurch neue Kartenspiele ohne Anpassung des Frameworkkerns integriert werden können.

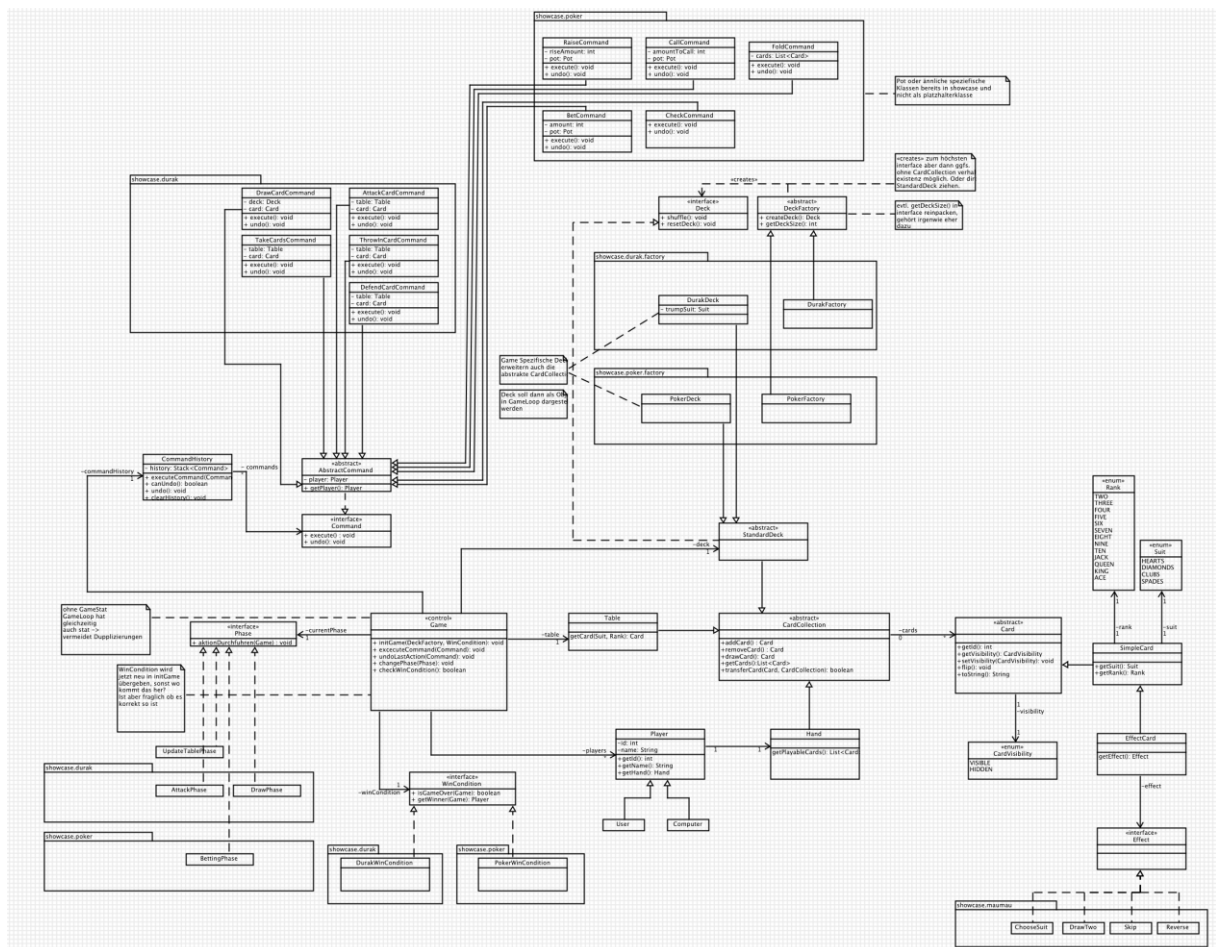


Abbildung 4.8: Gesamt-Klassendiagramm

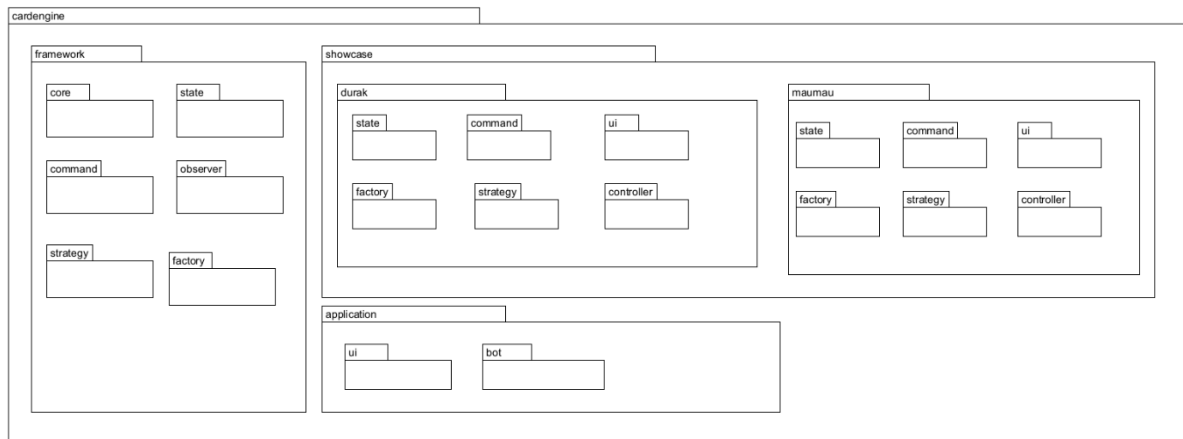


Abbildung 5: Package Struktur

5 Implementierung

In diesem Kapitel werden die wichtigste Ergebnisse der Implementierung des Card Game Engines vorgestellt.

Das Projekt ist in allgemeine Framework Komponenten und spielspezifische Implementierungen (Showcases) aufgeteilt. Die erste Version von der Struktur unterscheidet sich von der Abbildung, da wir die KI-generierte Inhalte entkoppeln wollten, und die Klasse Control lag im Application package, jetzt ist das nicht die Struktur. Die Grundidee hinter der Struktur (Abbildung 5.9.) ist die Umsetzung von Inversion of Control, um die zyklische Paketunabhängigkeit zu gewährleisten. Framework $\leftarrow$ Showcase $\leftarrow$ Anwendung

5.1 Aufbau des Frameworks

Das Package core enthält zentrale Modellklassen wie Game, Card, EffectCard, Player, CardCollection und StandardDeck. Die Klasse Game verwaltet den aktuellen Spielzustand und dient als Orchestra. Dazu gehören die Spieler, das Deck, die Karten auf dem Tisch, und die aktuelle Spielphase. Weitere Framework-Stellen sind nach ihrer Aufgabe in eigene Packages aufgeteilt

command	die Verarbeitung und Speicherung von Spielaktionen
state	Spiegelt die Spielphasen
strategy	austauschbare Regeln und Effekte der Karten
factory	Erzeugung verschiedener Kartendecks
observer	Die Schnittstelle zur Benachrichtigung über Änderungen

Die konkrete Regeln für Durak und Mau Mau sind außerhalb von dem Framework in den jeweiligen Showcase Packages. Das Framework bleibt dadurch von den konkreten Showcases

unabhängig. Ein neues Kartenspiel kann ergänzt werden, indem Phasen, Commands, Strategien, Fabriken implementiert werden.

Die implementierte Struktur sieht aus wie folgt:

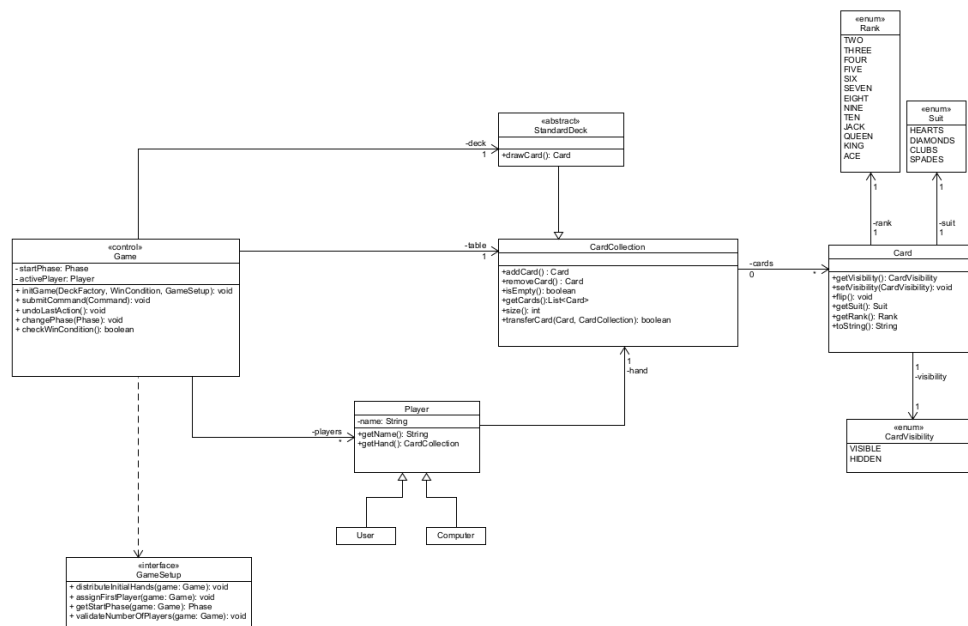


Abbildung 5.1: Basis-Klassendiagramm vereinfacht

Die Klasse Game bildet das zentrale Steuerungsobjekt und verwaltet das Deck, den Tisch, die Spieler und den aktiven Spieler. Karten werden in CardCollection-Objekten gespeichert, die sowohl für den Tisch als auch für die Hand eines Spielers verwendet werden. StandardDeck erweitert CardCollection und ergänzt die Funktion zum Ziehen einer Karte. Jede Card besitzt einen Rank, einen Suit und eine CardVisibility. Player kann durch User und Computer spezialisiert werden, während beide Varianten dieselbe grundlegende Spielerstruktur verwenden. Das Interface GameSetup entkoppelt die Initialisierung vom allgemeinen Framework und ermöglicht unterschiedliche Startregeln für verschiedene Kartenspiele

In der ersten Version wurden mit Hand, Table und SimpleCard mehrere zusätzliche Klassen verwendet. In der überarbeiteten Version wurden diese Strukturen durch die allgemeinere Klasse CardCollection ersetzt, wodurch unnötige Spezialisierungen entfernt wurden. Zusätzlich wurden Game und GameSetup in das Modell aufgenommen, um den Spielzustand und die Initialisierung verschiedener Spiele zentral abzubilden. Dadurch ist die neue Struktur einfacher und besser erweiterbar.

5.2 Initialisierung eines Spiels

Die zentrale Initialisierung erfolgt über die Methode initGame() der Klasse Game. Diese Objekte enthalten die veränderlichen Teile der Spielkonfiguration. Zuerst erzeugt die übergebene DeckFactory das Deck. Danach wird die Anzahl der Spieler durch das GameSetup geprüft. Das Deck wird gemischt und die Anfangskarten werden an die Spieler verteilt.

Anschließend bestimmt der GameSetup den ersten aktiven Spieler und erzeugt die Startphase. Für Durak wird DurakGameSetup verwendet, Mau-Mau wird durch MauMauSetup initialisiert. Beide Klassen implementieren dasselbe Interface GameSetup.

Nach der Initiatisierung wird die Methode start() aufgerufen. Dabei wird die Startphase aktiviert und der aktuelle Spielzustand an die registrierten Listener gemeldet.

5.3 Verarbeitung eines Zuges (Stanislav)

Ein Zug wird durch die Methode submitCommand() der Klasse Game verarbeitet. Die Methode erhält ein Command-Objekt, das die gewünschte Spielaktion enthält. Ist keine aktuelle Phase vorhanden oder wurde kein Command übergeben, wird die Verarbeitung beendet.

Anschließend prüft die aktuelle Phase mithilfe der Methode isValid(), ob die Aktion in der aktuellen Spielsituation erlaubt ist. Ein ungültiger Zug wird nicht ausgeführt. Stattdessen werden die registrierten GameListener über onInvalidMove() benachrichtigt.

Ein gültiges Command wird durch den CommandHistory ausgeführt und zusammen mit der aktuellen Phase sowie mit dem aktiven Spieler gespeichert. Danach bestimmt die Methode next() der aktuellen Phase den nächsten Zustand. Die Klasse Game übernimmt die neue Phase und prüft anschließend mithilfe der verwendeten WinCondition, ob das Spiel beendet ist.

Ist das Spiel beendet, wird die aktuelle Phase auf null gesetzt und der ermittelte Gewinner über onGameOver() gemeldet. Nach Abschluss der Verarbeitung werden die Listener durch onStateChanged() über den neuen Spielzustand informiert.

Durch diesen Aufbau bleiben die eigentliche Spielaktion und die zugehörigen Regeln voneinander getrennt. Ein Command bewegt beispielsweise eine Karte, während die Phase entscheidet, ob diese Bewegung erlaubt ist und welcher Zustand anschließend folgt.

Die Controller von MiniGame, Durak und Mau-Mau erzeugen die jeweils passenden Commands und übergeben sie über denselben Ablauf an die Klasse Game.

5.4 Umsetzung der Patterns

5.4.1 State Pattern (Lukas)

Zuerst wurde dem Framework das Interface hinzugefügt vergleiche Code Snippet 5.4.1.1. Dieses Interface wird von der Game Klasse als currentPhase gehalten und gibt den implementierenden Spielen vor nach welcher Form eine Phase aufgebaut sein muss (Vertrag).

```
public interface Phase {  
    boolean isValid(Game game, Command cmd);  
    Phase next(Game game, Command cmd);  
}
```

Code Snippet 5.4.1.1: Basis-Klassendiagramm vereinfacht

Der Code Snippet 5.4.1.2 zeigt wie die Controller Klasse die Phasen verwaltet, den Command mithilfe der Phase validiert und in die nächste Phase übergeht.

```
public void submitCommand(Command command) {
    if (currentPhase == null || command == null) return;

    if (!currentPhase.isValid(this, command)) {
        notifyInvalidMove(command);
        return;
    }

    commandHistory.executeCommand(command,
currentPhase, activePlayer);

    Phase nextPhase = currentPhase.next(this, command);
    changePhase(nextPhase);

    if (checkWinCondition()) {
        changePhase(null);
        notifyGameOver(getWinner());
    }
    notifyStateChanged();
}
```

Code Snippet 5.4.1.2: Submit Command

Eine Gemeinsamkeit aller implementierten Phasen ist das instanceof Konstrukt in den isValid()-Methoden, vergleiche Beispiel für DefendPhase.java Code Snippet 5.4.1.3

```
if (cmd instanceof TakeCardCommand take) {
    return take.getPlayer() == verteidiger;
}
```

Code Snippet 5.4.1.3: Submit Command

So kann für jede Phase festgelegt werden welche Aktionen (Commands) in dieser Phase überhaupt möglich sind und welche weiteren Bedingungen erfüllt sein müssen für eine korrekte Verwendung.

5.4.2 Command Pattern (Stanislav)

Im Framework wurde das Interface Command definiert. Jedes Command muss eine Aktion ausführen und diese wieder rückgängig machen können. Code Snippet X.

```
public interface Command {  
    void execute();  
    void undo();  
}
```

Code Snippet 5.4.2.1: Interface Command

Die abstrakte Klasse AbstractCommand implementiert dieses Interface und speichert den Spieler, der die Aktion ausführt. Konkrete Commands müssen dadurch die Spielvariable nicht selbst implementieren. Vor der Ausführung wird der Command durch die aktuelle Phase validiert. Die Phase entscheidet, ob die Aktion in der aktuellen Spielsituation erlaubt ist. Dadurch werden die Spielregeln und die Durchführung einer Aktion getrennt. Gültige Commands werden von der Klasse CommandHistory ausgeführt und gespeichert.

```
public void executeCommand(Command command, Phase phasebefore, Player  
activePlayerBefore) {  
    if (command != null) {  
        command.execute();  
        history.push(new HistoryEntry(command, phasebefore,  
activePlayerBefore));  
    }  
}
```

Code Snippet 5.4.2.2: Execute Command Methode

Das Problem wurde jedoch erkannt. Die Undo-Methode macht die Aktion zwar rückgängig, funktioniert aber nicht korrekt, weil sie nicht weiß, welcher Spieler gerade aktiv ist und welche Phase aktuell läuft. Die Aktion wurde rückgängig gemacht, aber das Spiel ist bereits beim anderen Spieler, was zu einem fehlerhaften Undo führt. Deshalb wurde entschieden, eine Klasse zu erstellen, die diese Informationen enthält. Das heißt, HistoryEntry dient als Memento.

```
public final class HistoryEntry {  
    private final Command command;  
    private final Phase phaseBefore;  
    private final Player activePlayerBefore;  
}
```

Code Snippet 5.4.2.3: HistoryEntry (Memento für CommandHistory)

Beim Aufruf von undo() wird der letzte Eintrag vom Stack entfernt und die Methode undo() des gespeicherten Command ausgeführt. Die Klasse Game stellt die vorherige Phase und den vorherigen aktiven Spieler wieder her.

Die aktuelle Undo-Funktion stellt einfache Kartenbewegung und den vorherigen Phasenzustand wieder her. Auf diese Weise können neue Spielaktionen ergänzt werden, ohne die zentrale Verarbeitung in Game grundsätzlich zu verändern. Für Durak existieren beispielsweise AttackCardCommand, DeffendCardCommand, TakeCardCommand, ThrowInCardCommand,

DrawCardCommand und EndAttackCommand. Mau-Mau verwendet unter anderem PlayCardCommand und DrawCardCommand. Siehe Abbildung 5.4.2

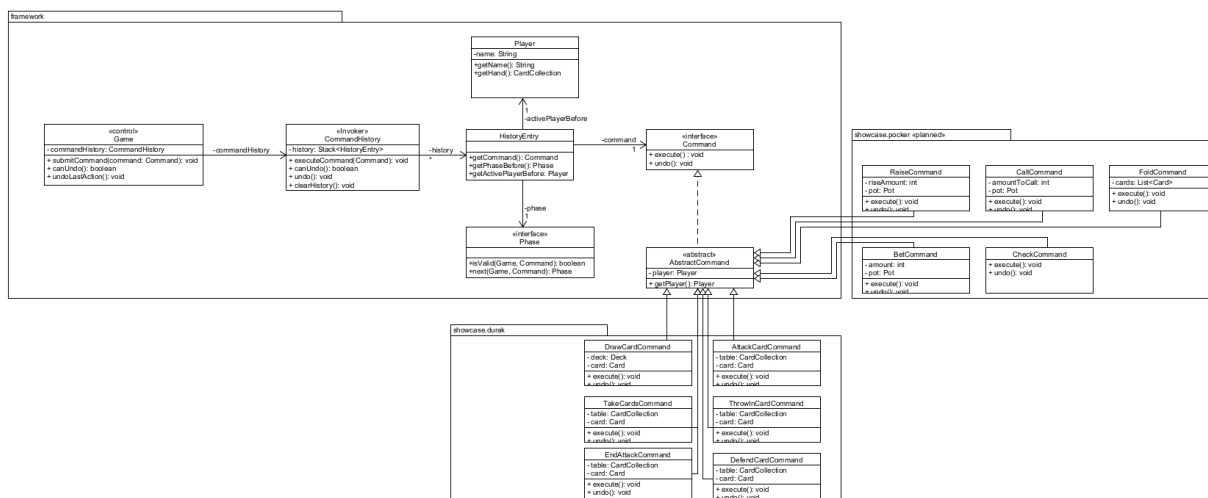


Abbildung 5.4.2: Command Pattern Implementierung

5.4.3 Strategy (Stanislav)

Das Strategy Pattern wird für austauschbare Spielregeln verwendet. Das Interface WinCondition legt mit isGameOver() und getWinner() fest, wie das Spielende geprüft und ein Spieler bestimmt wird. Code Snippet 5.4.3.1 Bei der Initialisierung wird die konkrete Strategie über initGame() übergeben. Dadurch muss die zentrale Spielverarbeitung die Regeln der einzelnen Kartenspiele nicht kennen.

```
public interface WinCondition {
    boolean isGameOver(Game game);
    Player getWinner(Game game);
}
```

Code Snippet 5.4.3.1: WinCondition

Für die Showcases existieren `DurakWinCondition` und `MauMauWinCondition`. `DurakWinCondition` beendet das Spiel erst bei leerem Deck und einem einzigen Spieler, der immer noch in der Hand Karten hat. `MauMauWinCondition` prüft zuerst, ob ein Spieler keine Handkarten mehr besitzt. Bei einem leeren Deck wird ersatzweise der Spieler mit den wenigsten Karten bestimmt.

Eine weitere Strategie wird durch das Interface `Effect` beschrieben. Es enthält die Methode `apply(Game)`. Code Snippet 5.4.3.2

```
public interface Effect {
    void apply(Game game);
}
```

Code Snippet 5.4.3.2: Interface Effekt

EffectCard erweitert Card und speichert eine solche Strategie. Im MauMauDeck haben bestimmte Karten folgende Effekte:

SEVEN	DrawTwoEffect	Der nächste Spieler nimmt 2 Karten aus der Deck
EIGHT	SkipEffect	Der Zug bei dem nächsten Spieler wird übersprungen
JACK	ChooseSuitEffect (KI generiert)	Der Spieler kann auswählen, welcher Suit als nächstes gelegt werden soll. (Hearts, Diamonds, Clubs, Spades)

Tabelle 5.4.1: Karten mit ihren Effekten

Dadurch bleiben die einzelnen Karteneffekte in eigenen Klassen und müssen nicht als lange Fallunterscheidung in der Phase oder in Game stehen. Die Effekte werden den Karten einmalig bei ihrer Erzeugung zugewiesen. Ein Austausch der Strategie zur Laufzeit, beispielsweise über eine Methode wie `setStrategy()`, wäre grundsätzlich möglich, ist in der aktuellen Implementierung jedoch nicht vorgesehen. Die beiden Klassendiagramme dieser Patterns wurden ohne Änderungen gegenüber dem ursprünglichen Entwurf implementiert.

5.4.4 Factory (Stanislav)

Die Erklärung im Folgenden erfolgt nach Refactoring Guru [RefGuru] und die Implementierung nach [KleuFolien]

Für die Erzeugung der Kartendecks wird das Factory Pattern eingesetzt. Die abstrakte Klasse DeckFactory (vgl. Code Snippet 5.4.4.1) definiert die Methoden `createDeck()` und `getDeckSize()`. Game erhält bei der Initialisierung nur eine DeckFactory und arbeitet anschließend mit dem erzeugten Deck über das Interface Deck (vgl. Code Snippet 5.4.4.2). Dadurch enthält Game keine Abhängigkeit zu einem konkreten Kartenspiel oder einer festen Deckgröße.

```
public abstract class DeckFactory {
    public abstract Deck createDeck();
    public abstract int getDeckSize();
}
```

Code Snippet 5.4.4.1: die Klasse DeckFactory

```
public interface Deck {
    void shuffle();
    Card drawCard();
    boolean isEmpty();
    int getDeckSize();
    int getStartIndex(Rank[] ranks);
}
```

Code Snippet 5.4.4.2: Interface Deck

Durak-Deck-Factory erzeugt ein Durak-Deck mit 36 Karten. Das Deck enthält die Ränge Sechs bis Ass in allen vier Farben. DurakDeck überschreibt shuffle(), damit nach dem Mischen die unterste Karte aufgedeckt und ihre Farbe als Trumpffarbe gespeichert wird.

Mau-Mau-Deck-Factory erzeugt ein Mau-Mau-Deck mit 32 Karten von Sieben bis Ass. Bei der Erzeugung entscheidet MauMauDeck außerdem, ob eine normale Card oder eine EffectCard angelegt wird. Den Rängen Sieben, Acht und Bube werden dabei die passenden Effektstrategien zugeordnet.

Die konkreten Decks erben von StandardDeck. Diese abstrakte Klasse erweitert CardCollection und implementiert Deck. Sie stellt die gemeinsamen Funktionen shuffle(), drawCard() und getDeckSize() bereit. Die konkreten Fabriken müssen daher nur festlegen, welches Deck mit welcher Kartenanzahl erzeugt wird.

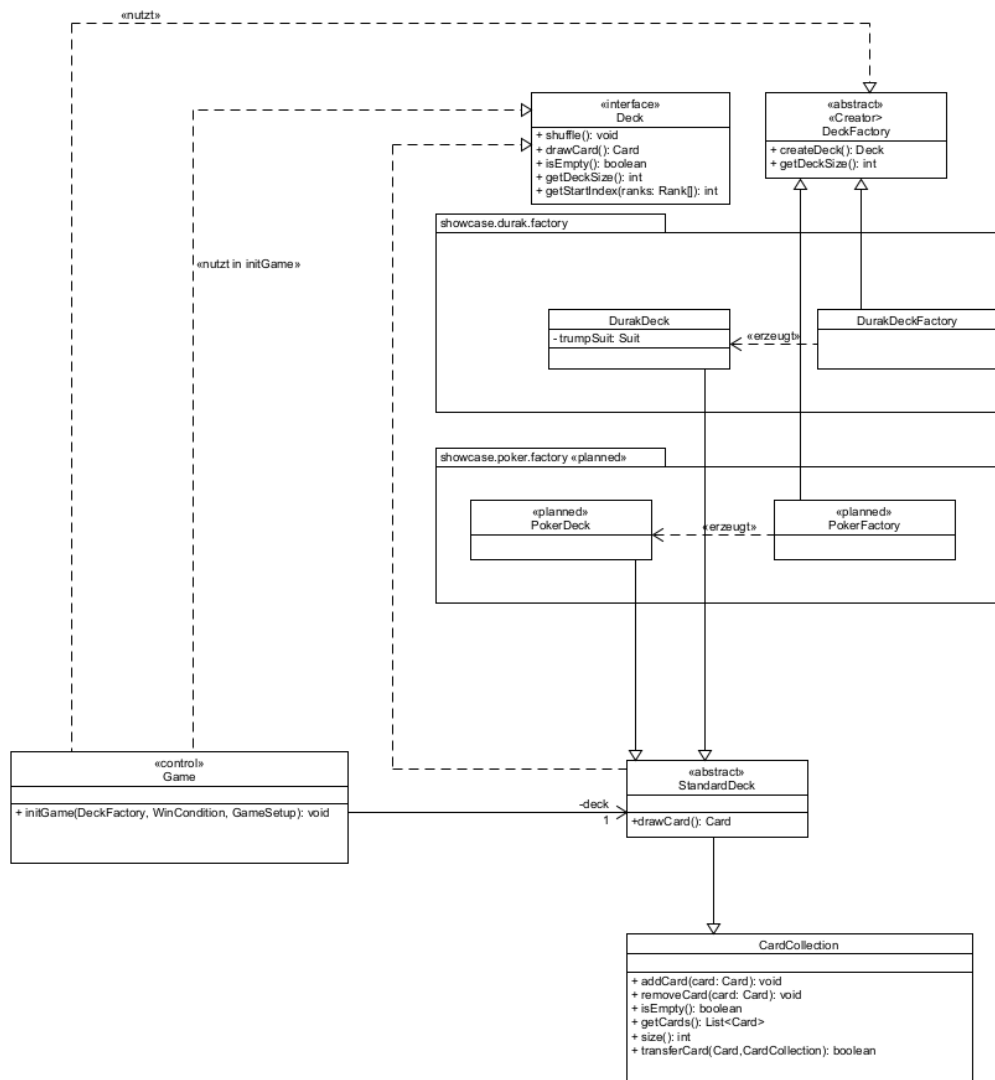


Abbildung: 5.4.4: Factory Klassendiagramm

5.4.5 Observer (Lukas)

Um das Observer Pattern umzusetzen, wurde das Beobachter Interface `GameListener` (vgl. Code Snippet 5.4.5.1) implementiert.

```
public interface GameListener {  
    void onStateChanged(Game game);  
    void onGameOver(Player winner);  
    void onInvalidMove(Command cmd);  
}
```

Code Snippet 5.4.5.1: Interface GameListener

Die Controller Klasse `Game` hält eine Liste mit allen `GameListener`n und eine Reihe an Hilfsmethoden, die die GUI darüber informieren das in einer anderen Phase übergegangen wurde. Vergleiche dazu Code Snippet 5.4.5.2.

```
public void addGameListener(GameListener l) {  
    listeners.add(l);  
}  
  
private void notifyStateChanged() {  
    for (GameListener l : new ArrayList<>(listeners)) {  
        l.onStateChanged(this);  
    }  
}  
  
private void notifyGameOver(Player winner) {  
    for (GameListener l : new ArrayList<>(listeners)) {  
        l.onGameOver(winner);  
    }  
}  
  
private void notifyInvalidMove(Command cmd) {  
    for (GameListener l : new ArrayList<>(listeners)) {  
        l.onInvalidMove(cmd);  
    }  
}
```

Code Snippet 5.4.5.2: Methoden, die Observer Pattern betreffen

Alle weiteren Verwendungen sind ausschließlich GUI bzw. Showcase Controller seitig, bei welchem sich dazu entschieden wurde in diesem Projekt keinen Fokus zu legen und daher mit Künstlicher Intelligenz generiert wurde und hier keine weitere Nennung finden soll.

5.5 Showcases (Stanislav)

5.5.1 Durak

Der Durak-Showcase nutzt ein gemischtes Deck mit 36 Karten. Die unterste Karte bestimmt den Trumpf, bleibt sichtbar im Deck und wird zuletzt gezogen. Zu Beginn erhält jeder Spieler sechs Karten. Der Spieler mit der niedrigsten Trumpfkarte beginnt, andernfalls der erste Spieler der Liste.

Der Ablauf besteht aus Angriffs-, Verteidigungs- und Ziehphase. Beim Angriff darf nur eine Karte mit einem bereits ausliegenden Rang ergänzt werden. Eine Verteidigung ist erfolgreich, wenn die Karte höher und gleichfarbig ist oder als Trumpf eine Nicht-Trumpfkarte schlägt. Andernfalls nimmt der Spieler alle Tischkarten auf. Nach einem erfolgreichen Angriff werden sie entfernt.

In der Ziehphase werden die Hände auf sechs Karten ergänzt. Spieler ohne Karten scheiden bei leerem Deck aus der Rollenverteilung aus. Das Spiel endet, sobald das Deck leer ist und höchstens ein Spieler noch Karten besitzt. Dieser Spieler ist der Durak.

Unterstützt werden menschliche Spieler und einfache Bots, deren mögliche Commands von der aktuellen Phase geprüft werden.

5.5.2 Mau-Mau

Der grundlegende Mau-Mau-Showcase mit Setup, Phase, Commands und Benutzeroberfläche wurde mit künstlicher Intelligenz generiert. Das Spiel verwendet ein Deck mit 32 Karten. Jeder Spieler erhält fünf Karten und eine offene Startkarte wird auf den Tisch gelegt. In der PlayPhase darf der aktive Spieler eine passende Karte ablegen oder eine Karte vom Deck ziehen.

Die Sonderkarten verwenden unterschiedliche Effekte. Bei einer Sieben zieht der nächste Spieler zwei Karten, eine Acht überspringt den nächsten Spieler und bei einem Buben wird eine neue Farbe gewählt. siehe. 5.4.3

Das Spiel endet, sobald ein Spieler keine Karten mehr besitzt. Ist das Deck leer, gewinnt der Spieler mit den wenigsten Karten.

5.5.3 Minigame

Das MiniGame wurde als einfacher Test-Showcase für das Framework erstellt. Es dient dazu, das Zusammenspiel von GameSetup, Phase, Command, Factory, WinCondition und Observer mit möglichst einfachen Regeln zu überprüfen. In jedem Zug zieht der aktive Spieler eine Karte, danach ist der nächste Spieler an der Reihe. Das Spiel endet, sobald ein Spieler sechs Karten besitzt oder das Deck leer ist. Der gesamte MiniGame-Showcase wurde mit künstlicher Intelligenz generiert.

6 Validierung

Nach Abschluss der Entwicklungsphase sind einige Anforderungen nicht vollständig umgesetzt worden auf welche im Weiteren eingegangen werden soll. Außerdem wurden einige Maßnahmen zur Qualitätssicherung angewandt welche hier vorgestellt werden sollen.

6.1 Erfüllung der Anforderungen (Akim)

ID	Anforderung (aus Rupp-Schablone)	Umsetzende Klasse(n) / Interface	Realisierende Methode(n)
A01	Nach dem das System gestartet wurde soll das System ein Deck anlegen	DeckFactory, Game	DeckFactory.createDeck(), Game.initGame()
A02	Nach dem Anlegen des Decks soll das System dieses mischen	Deck, StandardDeck	Deck.shuffle(),
A03	Nach dem das Deck gemischt wurde soll das System die Hand austeilen	GameSetup	GameSetup.distributeInitialHands()
A04	Sobald alle Player eine Hand haben, soll das System die Reihenfolge der Player bestimmen.	GameSetup,	GameSetup. assignFirstPlayer(),
A05	In jeder Round vor dem Turn des ersten Players soll das System die Möglichkeit haben den Table zu aktualisieren	GameListener	GameListner.notifyStateChanged()
A06	Nach dem der Table und Hand vorbereitet sind soll das System dem aktiven Player die Möglichkeit bieten	Game	Game.start(), Game.submitCommand()

	seinen Turn durchzuführen		
A07	Wenn ein Turn begonnen wird, soll das System dem Player die Möglichkeit bieten die Phasen zu durchlaufen	Phase, Game	Game.start(), Game.changePhase(), Phase.next()
A08	Wenn eine Phase begonnen wird, soll das System dem Player die Möglichkeit bieten Actions durchzuführen	Command, Game	Command.execute(), Game.submitCommand()
A09	Wenn eine Action durchgeführt wird, soll das System prüfen, ob die Action gültig ist	Game, Phase	Game.submitCommand() Phase.isValid()
A10	Ist die Action ungültig so soll das System dem Player die Möglichkeit bieten eine andere Action durchzuführen	Game, GameListener, Phase	Game.notifyInvalidMove() GameListener.onInvalidMove() Phase.isValid()
A11	Ist die Action gültig so soll das System prüfen, ob eine weitere Aktion möglich ist	Phase	Phase.next()
A12	Ist eine weitere Aktion möglich so soll das System dem Player die Möglichkeit bieten eine weitere Action durchzuführen	Phase, Command, Game	Phase.next() Command.execute(), Game.submitCommand()
A13	Kann der Player eine weitere Aktion	Command, EndAttackCommand	Command.execute(),

	durchführen soll das System dem Player die Möglichkeit bieten das Abzulehnen und in die nächste Phase zu gehen		
A14	Ist die Action gültig so soll das System dem User (menschlicher Player) die Möglichkeit bieten ein Undo (Action rückgängig) zu machen	CommandHistory, Game	CommandHistory.canUndo(), CommandHistory.undo() Game. undoLastAction()
A15	Versucht der User ein Undo zu machen prüft das System, ob ein Undo möglich ist	CommandHistory,	CommandHistory.canUndo(),
A16	Ist ein Undo möglich so soll das System dem User die Möglichkeit bieten eine neue Action durchzuführen	Command, Game	Command.execute(), Game.submitCommand()
A17	Ist ein Undo nicht möglich so soll das System dem User das anzeigen und die Möglichkeit bieten eine weitere Aktion durchzuführen	Command, Game, GameView,	Command.execute(), Game.submitCommand() GameView.log()
A18	Am Ende einer Phase soll das System prüfen, ob eine Win Condition erfüllt ist.	WinCondition, Game	WinCondition.isGameOver(), Game.checkWinCondition()

A19	Ist eine Win Condition erfüllt soll das System den Winner anzeigen	WinCondition, Game, GameView	WinCondition.getWinner(), Game.notifyGameOver() GameView.showGameOver()
A20	Nach dem Prüfen einer Win Condition soll das System die nächste Phase einleiten	Game, PhaseState	Game.changePhase(), Phase.next()
A21	Ist keine nächste Phase einleitbar (letzte Phase) soll das System den Turn des Players beenden	DrawPhase, DurakTurn	DrawPhase.next(), DurakTurn.startAttack()
A22	Nach beenden des Turns soll das System prüfen, ob ein nächster Player seinen Turn beginnen darf	Es ist Relikt aus alter Modellierung des Aktivitätsdiagramm der GameLoop	Das ist schlecht formulierte Anforderung, weil das einfach A21 wiederholt und mit der Phase.next() schon abgedeckt ist.
A23	Darf kein nächster Player seinen Turn beginnen so soll das System die nächste Round starten	Siehe A22	

Tabelle 6.1.1: Anforderungen erfüllt

6.1.1 Framework (Stakeholder Spielentwickler) (Stanislav)

Das Ziel verschiedene Kartenspiele auf einer gemeinsamen Basis umzusetzen, wurde grundsätzlich erreicht. Das Framework stellt mit Card, Player, CardCollection, StandardDeck und Game zentrale Modelklassen bereit. Die vorgesehenen Entwurfsmuster sind umgesetzt. Die Trennung zwischen Framework und konkreten Spielen ist gelungen. Für neue Spiele müssen eigene Setups, DeckFactories, WinConditions, Commands und Phasen erstellt werden. Die Spielregeln sind nicht in einer eigenen zentralen Klasse definiert, sondern auf die jeweiligen Phasen und Commands verteilt. Das Framework stellt die grundlegende Struktur für rundbasierte Kartenspiele bereit. Die spielspezifische Logik müssen in Showcase umgesetzt werden.

6.1.2 Showcase (Stakeholder User) (Akim)

Neben der technischen Überprüfung der Framework-Architektur muss die Funktionsfähigkeit des Showcases aus der Sicht des Endanwenders, also des Stakeholders User, evaluiert werden.

Der Anwender erwartet eine intuitive Spielsteuerung, die korrekte Durchsetzung der Spielregeln, transparente Rückmeldungen vom System sowie die Möglichkeit, den Prozess zu steuern und Aktionen gegebenenfalls rückgängig zu machen.

Die Validierung erfolgte anhand der prototypischen Showcase Anwendungen Durak, Mau-Mau und MiniGame durch die Gegenüberstellung des Use-Case-Diagramms und der Rupp-Schablone mit den eigentlichen Realisierungen im Quellcode. Die Showcase Anwendungen demonstrieren erfolgreich, dass alle benutzerrelevanten Anforderungs-Tracing A01–A21 aus Sicht des Stakeholders User erfüllt und bedienbar sind.

Die ursprünglichen Anforderungen A22 und A23 verloren durch den Architekturwechsel hin zu einem eventbasierten GUI-Ansatz (`submitCommand()` und `GameListener`) ihre direkte Relevanz. Der Phasen- und Spielerwechsel erfolgt nun reaktiv und zustandsgesteuert nach jeder eingereichten Aktion über `Phase.next()`.

6.1.3 Nicht erfüllte Anforderungen

Im Folgenden werden sich die nicht erfüllten Anforderungen weiterer Betrachtung unterzogen.

6.1.3.1 Zuleger

Im Laufe der Implementierung des Frameworks für den Durak Showcase ist aufgefallen das das Design das Ermöglichen einem anderen Spieler eine Aktion durchzuführen innerhalb von einem Zug des aktiven Spielers nicht unterstützen kann.

Konkrete Ursache dafür ist, dass `AttackPhase.isValid` akzeptiert einen `ThrowInCardCommand` des Zulegers, aber `activePlayer` bleibt immer der Angreifer. Weder GUI noch Bot können den Command deshalb überhaupt einreichen.

Die Lösung würde die Rotation Zustand pro Angriff erfordern, was nur in einem Durak-Spezifischen Kontext sauber unterzubringen wäre.

6.1.3.2 Eine offene Angriffskarte

Um nicht noch weiter von den ersten Designentscheidungen zu entfernen, wurde sich dagegen entschieden mehrere offene Angriffskarten zu erlauben, was in dem Kartenspiel normalerweise erlaubt ist, vorallem zum Ende nachdem sich der Verteidiger dazu entschieden hat aufnehmen müssten Angreifer weitere Gelegenheit bekommen Karten zuzulegen, was aber nicht von dem Durak Showcase implementiert wurde

6.1.3.3 Ziehreihenfolge beim Nachziehen

Durak schreibt Angreifer → Zuleger → Verteidiger vor; implementiert ist jedoch lediglich die Sitzreihenfolge

6.1.3.4 Undo bei komplexen Effekten

Komplexe Folgen eines Commands, beispielsweise bereits ausgeführte Karteneffekte, werden noch nicht in jedem Fall vollständig rückgängig gemacht. Technisch wäre es möglich, in dem Projekt ist das aber nicht vorgesehen.

6.2 Qualitätssicherungsmaßnahmen

6.2.1 Konsistenzanalyse (Iteration 1 → 2)

Der Code, die UML und Doku wurden systematisch gegeneinandergehalten und es wurden die Befunde wie Konsolen-gameLoop / Pattern-Ausbau / Event-Umbau identifiziert und darauffolgend im Code so umgebaut das es funktionstüchtig wird sowie eine neue Iteration an UML wurde dafür angefertigt.

6.2.2 Test-First mit JUnit

Es wurde getestet, wie sich die Entwicklung verbessert bzw. Verändert, wenn zuvor klare Anforderungen an Künstliche Intelligenz weitergegeben werden und diese darauf basierend eine Testklasse generiert. In diesem Zuge entstand DurakStatePatternTest. Allerdings hat sich das eher als sinnfrei herausgestellt, weil die Testklasse einfach nicht genug Testfälle angeboten hat und trotz fertiger Implementierung der Testvorgaben, große Teile der wirklich Notwendigen Funktionalität gefehlt hat.

6.2.3 KI-gestützte Architekturanalysen

An einigen Stellen während der Entwicklung wurden KI-Modelle befragt, ob die Architektur gemäß der Dokumentation und UML sind und inwieweit die Abweichungen Sinnvoll sind.

7 Abschlussbetrachtungen

Abschließend soll noch einmal die gesamte Hausarbeit betrachtet werden.

7.1 Zusammenfassung (Lukas)

Ziel der Vorliegenden Hausarbeit ist die Entwicklung eines objektorientierten Frameworks für rundenbasierte Kartenspiele in Java. Zentral war die Beobachtung die Kartenspiele wie Mau Mau oder Durak zwar unterschiedliche Regeln aber zentrale, strukturelle Gemeinsamkeiten aufweisen wie bspw. Kartenverwaltung in Händen und Decks sowie das Aufteilen der Spielzüge in Phasen. Ziel war dementsprechend den wiederverwendbaren Kern als Framework umzusetzen und einen Showcase eines komplizierteren Kartenspiels das dieses Verwendet zu entwickeln.

Die Entwicklung erfolgte eng an den den in der Vorlesung vorgestellten Vorgehen [Kleu]. In der Anforderungsanalyse wurden zunächst ein Glossar sowie Stakeholder und Ziele festgelegt, daraus wurden dann Use Cases mit zugehöriger Dokumentation abgeleitet sowie die Anforderungen nach der Rupp-Schablone formuliert und anschließend die Aktivitätsdiagramm.

Fortgefahren wurde mit dem Grobdesign im Zuge welcher ein Aktivitätsdiagramm sowie Klassendiagramme für das State Pattern für die Spielphasen, das Command Pattern für Spielzüge und deren Rücknahme, das Factory Method Pattern für die Deckerzeugung sowie das Strategy Pattern für Siegbedingungen und Karteneffekte. Das Observer Pattern kam in einer zweiten Iteration hinzu, nachdem die ursprünglich vorgesehene Konsolenschleife durch eine ereignisgesteuerte Oberfläche ersetzt wurde.

Die Implementierung erfolgte in zwei Ebenen zuerst wurde das Framework entwickelt, welches zuerst aus den Klassendiagrammen abgeleitet wurde und dann im nächsten Schritt mit der notwendigen Logik bestückt wurde. Als Showcases entstanden Durak, Mau-Mau und ein minimales Testspiel zur Absicherung des Zusammenspiels aller Muster.

Weiter mit der Validierung erfolgte über ein Tracing sämtlicher Anforderungen sowie einer Konsistenzanalyse zwischen UML und Quelltext in welchem Zuge die zweite Iteration der UML-Dateien erstellt wurden. Der überwiegende Teil der Anforderungen wurde erfüllt.

7.2 Ausblick (Lukas)

Aus den in Kapitel 6 dokumentierten Lücken betreffend der Zuleger-Rolle, mehrere gleichzeitig offene Angriffskarten, die Durak-spezifische Ziehreihenfolge sowie die vollständige Rücknahme komplexer Karteneffekte; die Ursachen lassen sich jeweils auf konkrete Entwurfsentscheidungen zurückführen und sind in Kapitel 6 dokumentiert. Hier soll erstmal nicht weiter auf Verbesserungen der Showcases eingegangen werden, sondern hinterfragt werden muss, ob das entwickelte Framework dem Entwickler genug Freiraum bietet um komplexe Kartenspiele wie Durak zu entwickeln oder ob nur einfachere Spieler mit diesem

Framework zu entwickeln ist. Die eigentliche Belastungsprobe für ein Framework ist schließlich die Aufnahme eines weiteren, andersartigen Spiels. Der bereits modellierte, aber nicht implementierte Poker-Showcase eignet sich hierfür besonders, da er mit Einsätzen und einem Pot Zustände einführt, die im aktuellen Kern nicht vorgesehen sind, und zeigen würde, ob das Framework tatsächlich zu starr ist.

7.3 Fazit (Lukas, Akim)

Rückblickend hätten wir bereits während des Modellierungsprozesses des Aktivitätsdiagramms und der Klassendiagramme feststellen müssen, dass wir eine Eventbezogene Version brauchen. Außerdem wurden an einigen Stellen zu komplex modelliert, wo wir in der zweiten Iteration uns dazu entschieden haben, einige leere Klassen ganz rauszustreichen und zu entfernen (wie bspw. Table oder Hand). Außerdem wurde das Framework mit drei konkreten Showcases im Kopf modelliert, und dennoch konnte eines davon nicht sauber nach den OOAD-Methoden damit umgesetzt werden, es ist sich zu fragen, ob Spiele, an die gar nicht gedacht wurde, gut mit unserem Framework umzusetzen sind. Zwei Ansätze sind gescheitert. Der erste war das testgetriebene Vorgehen mit einer per KI aus den Anforderungen generierten Testklasse: Die Tests liefen durch, deckten aber so wenige Fälle ab, dass die grüne Suite trotz erheblicher funktionaler Lücken ein falsches Sicherheitsgefühl erzeugte. Die Lehre daraus ist, dass Testfälle aus den Anforderungen und nicht aus dem bereits vorhandenen Code abzuleiten sind. Der zweite war die Zuleger-Rolle im Durak-Showcase, deren Unvereinbarkeit mit dem gewählten Turn-Modell erst während der Implementierung aufgefallen ist. Ein Aktivitätsdiagramm mit getrennten Verantwortlichkeitsbereichen je Rolle hätte diesen Konflikt bereits in der Analysephase sichtbar gemacht.

Trotz aller Herausforderungen während des systematischen Entwicklungsprozesses ist es uns gelungen, eine sehr effektive Teamarbeit zu organisieren. Selbst angesichts unterschiedlicher Zeitpläne und zeitlicher Möglichkeiten der Teilnehmer verlief die Entwicklung harmonisch und ohne Sackgassen. Im Vergleich zu vielen anderen Projekten hat sich unsere sehr klare und präzise Aufgabenteilung für das gesamte Team hervorragend bewährt. Obwohl zu Beginn eine gewisse Anspannung im Team zu spüren war, haben wir einen großen Schritt nach vorne gemacht, sodass am Ende alle mit der Kommunikation und dem Prozess selbst rundum zufrieden waren.

Eine wichtige Ergänzung war unser starkes Projektmanagementsystem auf der Notion-Plattform. Es sorgte für einen vollständigen Überblick über das Projekt, ein transparentes Management der Verantwortlichen und des Zeitaufwands sowie offene Diskussionszweige zu einzelnen Tickets. Dieser Ansatz erwies sich als äußerst erfolgreich. Etliche morgendliche Meetings ermöglichten es, Lösungen vorzustellen und mögliche Verbesserungen zu diskutieren, die schließlich erfolgreich in UMLs und im Code umgesetzt wurden.

Der Entwicklungsprozess hat zudem gezeigt, dass ursprüngliche Ideen, die in der Planungsphase ideal erscheinen, bei der Umsetzung unter Umständen erhebliche Anpassungen

erfordern. Einige Muster wurden geändert und pragmatisch umgesetzt, während andere infolge dieser Änderungen nicht mehr die suboptimale Lösung darstellten. Eine wichtige Erkenntnis war, dass die Einhaltung der Algorithmen aus den Vorlesungen und der Musterbeispiele es ermöglicht, den gesamten Planungs- und Modellierungsprozess relativ klar und fast ohne Rückgriff auf externe Ressourcen umzusetzen.

Schließlich haben wir durch diese Arbeit mehr Erfahrung und Verständnis für die systematische Entwicklung gewonnen und gelernt, den Einsatz generativer KI zu dosieren. Für Oberflächen und gleichförmigen Code war er hilfreich; bei Architekturfragen führte er zu Vorschlägen, die für sich genommen plausibel wirkten, aber nicht zu den zuvor getroffenen Entwurfsentscheidungen passten. Alle KI-gestützten Anteile sind daher im Bericht und im Quellcode gekennzeichnet.

Anhang 1 Spielregeln für Durak (KI)

Durak

Vorbereitung

- **36 Karten** nutzen (von 6 bis Ass).
- Jeder Spieler bekommt **6 Karten**.
- Die nächste Karte wird offen in die Mitte gelegt und bestimmt die **Trumpffarbe**. Der Rest kommt verdeckt darauf.

Spielablauf

- Der Spieler mit dem **kleinsten Trumpf** fängt an.
- Es wird reihum im **Uhrzeigersinn** gespielt.
- Ein Angreifer legt eine Karte. Der Verteidiger muss sie **schlagen** (höhere Karte gleicher Farbe oder ein Trumpf).
- Bei Erfolg kommen die Karten auf den Ablagestapel. Bei Misserfolg muss der Verteidiger alle Karten **nehmen**.
- Nach jedem Zug ziehen alle Spieler wieder bis auf **6 Karten** vom verdeckten Stapel nach

Anhang 2 Spielregeln für MauMau (KI)

MauMau

Vorbereitung

- Ein Standardblatt mit **32 Karten** nutzen (von 7 bis Ass).
- Jeder Spieler bekommt **5 bis 6 Karten** (je nach Vereinbarung).
- Die restlichen Karten bilden den verdeckten **Nachziehstapel**.
- Die oberste Karte wird offen danebengelegt und bildet den **Ablagestapel**.

Spielablauf

- Es wird reihum im **Uhrzeigersinn** gespielt.
- Man darf eine Karte ablegen, wenn sie in **Farbe** (z. B. Herz auf Herz) oder **Wert** (z. B. 8 auf 8) mit der obersten Karte übereinstimmt.
- Kann oder will man nicht legen, muss man **eine Karte ziehen**. Danach ist der nächste Spieler dran.

Sonderkarten

- **7 (Zwei ziehen)**: Der nächste Spieler muss 2 Karten ziehen, es sei denn, er legt selbst eine 7 (dann zieht der nächste Spieler 4).
- **8 (Aussetzen)**: Der nächste Spieler muss diese Runde überspringen.
- **Bube (Wünschen)**: Kann auf jede Karte gelegt werden. Der Spieler darf sich eine neue Farbe wünschen.

A Verantwortlichkeiten

Artefakt	Verantwortlich	Mitwirkend	
1 Einleitung	Akim		
2 Grundlagen	Akim 2, 2.2, 2.3, 2.5	Lukas 2.1, 2.4	Stanislav 2.5
3 Anforderungsanalyse	Akim	Lukas 3.6	
4 Modellierung:	Akim 4.3, 4.4, 4.5, 4.8	Lukas 4.2, 4.7	Stanislav 4.1, 4.6
5 Implementierung	Stanislav 5, 5.1, 5.2, 5.3, 5.4.2, 5.4.3, 5.4.4	Lukas 5.4.1, 5.4.5	
6 Validierung	Lukas 6.1.3, 6.2	Stanislav 6.1.3.4	Akim 6.1, 6.1.2
7 Fazit	Lukas 7.1, 7.2, 7.3	Akim 7.3	

Die Verantwortlichkeiten für die Implementierung stehen jeweils in der Dokumentation des Quellcodes.

B Abgabe

Hier steht der Inhalt des abgegebenen Zip-Verzeichnisses

Verzeichnis	Inhalt
/Hausarbeit	Diese Hausarbeit im Originalformat und als PDF
/Quellcode/CardGameEngine	Die entstandenen Projektergebnisse als Eclipse-Projekt
/Quellcode/CardGameEngine/docs	UML-Diagramme und weitere Doku zu dem Projekt

C Stundenzettel

Zur Projektverwaltung wurden Tickets verwendet, in denen die Aufgabe und der dafür aufgewendete Zeitaufwand erfasst wurden. Hier sind alle Tickets aller Teammitglieder in Form 3 Tabellen dargestellt.

Kategorie	Ticket name	Dauer (h)
2 Anforderungen	Use-Case-Diagramm	2.0
2 Anforderungen	Use-Case Documentation	1.5
2 Anforderungen	Stakeholder & Ziele	2.0
3 Architektur	Weekly Standup	2.0
3 Architektur	Zusammenbau von der Gesamt-Klassen-Diagramm	5.0
3 Architektur	Design Strategy Pattern (Sieg)	1.5
3 Architektur	Design Factory Pattern	2.0
3 Architektur	Design Command Pattern (Undo)	2.0
4 Code: Engine	Entities & Basis-Interfaces. UMLet in Projekt übertragen	4.0
6. Qualitätssicherung	Anforderungs-Tracing	3.0
7 Doku	Doku Akim	14.0
8 Finalisierung	Bericht anpassen	6.0
Akim Zdorovtsov	Insgesamt:	45.0h

Kategorie	Ticket name	Dauer (h)
0 Orga	Aufgabenpakete erarbeiten	1.0
0 Orga	Übersicht Projekt und Anforderungen analysieren	1.5
0 Orga	Zeitmanagement	1.5
0 Orga	E-Mail an Prof. Kleuker wegen Akim	2.0
1 Projekt-Setup	Grundregeln & Glossar	1.5
2 Anforderungen	Rupp-Schablone	1.5
2 Anforderungen	Aktivitätsdiagramm	2.0
2 Anforderungen	Use-Case-Diagramm	2.0
3 Architektur	Weekly Standup	2.0
3 Architektur	Design State Pattern für Phasen	1.5
4 Code: Engine	Evaluieren des derzeitigen Stands mit Claude	1.5
4 Code: Engine	Factory & Strategy Framework	1.0
4 Code: Engine	Observer-Pattern	2.0
4 Code: Engine	Game & GameLoop etc	3.5
4 Code: Engine	Command Pattern (Undo/Redo)	1.0
5 Code: Showcase	Command Pattern für Aktionen implementieren	4.0
5 Code: Showcase	State Pattern mit Phasen implementieren (und noch n paar andere Sachen)	9.5
5 Code: Showcase	User Interface (GUI)	4.0
5 Code: Showcase	Analyse und UI	1.0
5.1 Showcase Architektur	Aktivitätsdiagramm für Durak Controller Klasse	3.5
7 Doku	Doku Lukas	8.5
8 Finalisierung	Bericht anpassen	4.0
Lukas Ochs	Insgesamt:	60.0h

Kategorie	Ticket name	Dauer (h)
1 Projekt-Setup	IDE & Struktur einrichten	4.0
3 Architektur	Weekly Standup	1.0
3 Architektur	Basis-Klassendiagramm	5.0
3 Architektur	Design Strategy Pattern (Effekte)	5.0
5.1 Showcase Architektur	Showcase Übersicht + Klassendiagramm (Durak)	3.5
5 Code: Showcase	Spielspezifische Klassen	6.0
5 Code: Showcase	EffectCard Pattern in MauMau implementieren (optional)	3.0
5 Code: Showcase	Factory Pattern für Deckbauen	3.0
5 Code: Showcase	Integration & Testing	8.0
6 Qualitätssicherung	Finetuning der Docs (Feindesign 2)	3.0
6 Qualitätssicherung	Code-Review & Refactoring	4.0
6 Qualitätssicherung	Migration Projekt IntelliJ -> Eclipse	4.0
7 Doku	Projektabricht Analyse 1	2.0
7 Doku	Doku Stani	10.0
8 Finalisierung	Bericht anpassen	4.0
Stanislav Zholob	Insgesamt:	68.5h

D Erklärung

Hiermit versichern wir, dass wir diese Hausarbeit selbständig verfasst und keine anderen als die angegebenen Quellen und Hilfsmittel benutzt haben.



Datum: 03.08.2026

(Unterschrift)



Datum: 03.08.2026

(Unterschrift)



Datum: 03.08.2026

(Unterschrift)

Literaturverzeichnis

Weblinks zuletzt abgerufen am 02.08.2026.

[KleukAnf] S. Kleuker, Anforderungen an Abschlussaufgaben, <http://home.edvsz.hs-osnabrueck.de/skleuker/querschnittlich/AnforderungenAbschlussarbeit.pdf>

[KleukFolien] S Kleuker, Objektorientierte Analyse und Design, https://kleuker.iui.hs-osnabrueck.de/SoSe26_OOAD/OOAD_Folien.pdf

[Kleuk18] S. Kleuker, Grundkurs Software-Engineering mit UML, 4. aktualisierte Auflage, Springer Vieweg, Wiesbaden, 2018

[Kleuk19] S. Kleuker, Qualitätssicherung durch Softwaretests, 2. aktualisierte und erweiterte Auflage, Springer Vieweg, Wiesbaden, 2019

[RefGuru] <https://refactoring.guru/design-patterns/factory-method>

Ukrainisch-Deutsch Übersetzer: <https://www.deepl.com/en>